Online Learning in Open Data Space

Zhi Cao, Peijia Qin, Chin-Teng Lin, *Fellow, IEEE*, and Xin Yao, *Fellow, IEEE*

***Abstract*—In real-world data stream mining,instances are typically distributed in open data space where the composition of classes and features undergoes unpredictable changes, leading to the dual challenges of class evolution and feature evolution. Although several early studies simultaneously address both issues, they necessitate substantial data storage for model adaptation. Numerous subsequent studies focus on the development of online learning algorithms. However, these algorithms tackle the evolution in either feature space or label space, not both. In this paper, we propose a novel Ensemble of Passive-Aggressive Model (EPAM) to address challenges associated with the open data space in an online learning scenario. We initiate the research of online learning in the open data space by constructing several baseline models grounded in state-of-the-art online learning methods in the domains of class evolution and feature evolution, followed by an analysis of their limitations in handling real-world data streams. To overcome these limitations, EPAM incorporates a novel feature contributed bias classifier specifically designed for feature evolution, with the bias term capable of adapting to diverse feature spaces. Furthermore, a novel model adaptation strategy is developed to balance error feedback among classes designated as the negative class for each feature contributed bias classifier, therefore addressing the dynamic class imbalance induced by class evolution and enhancing multi-class classification performance. Comprehensive experiments on various synthetic and real-world data streams demonstrate the superior performance of EPAM.**

***Index Terms*—Data stream mining, online learning, open data space, class evolution, feature evolution.**

I. Introduction

REAL-WORLD data streams are characterized by their continuous generation of infinite length and perpetual evolution with unpredictable changes, for example, in the label space and the feature space, posing the challenges of class evolution [1], [2] and feature evolution [3], [4], [5]. In other words, both sets of classes and features are not fixed, with instances in the data stream distributed in the open data space. An illustrative example is depicted in Fig. 1. As the time passes, novel classes and features may emerge, existing ones may disappear, and disappeared ones may reoccur. This phenomenon occurs frequently in practical data streams. For example, in a social network like *Twitter*, new topics emerge while outdated topics are forgotten. Some forgotten topics such as the Olympic Games may regain popularity later. Similarly, the vocabulary in each tweet evolves over time, with new words being coined and older terms gradually falling out of use [3]. Furthermore, since the real-world data stream is generated continuously at large volumes and high velocities [6], storing and utilizing all historical data for model adaptation becomes impractical. Online learning, which processes instances one-by-one without storing past data for model adaptation, is well-suited to tackle these issues [7], [8]. While the absence of the requirement to store historical data enables online learning algorithms to swiftly adapt to changes in the data stream, it concurrently poses the challenge of tracking the evolving open data space and updating the model based solely on a single available data instance.

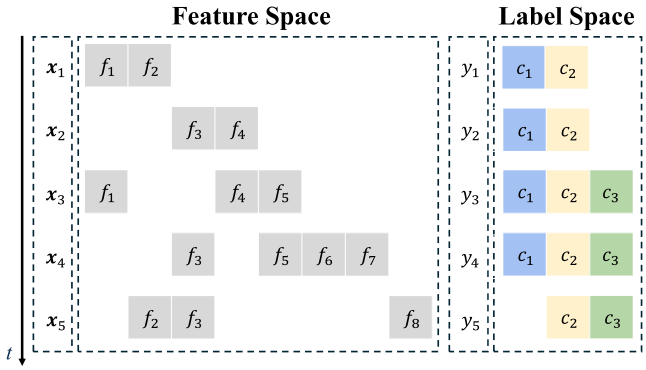


Fig. 1. An illustrative example of a data stream distributed in open data space. At time $t = 3$, within the feature space of $\boldsymbol{x}_3$, feature f_5 emerges while feature f_3 disappears. In the label space of y_3, the binary classification between classes c_1 and c_2 transitions to a three-class classification with the emergence of class c_3. By time $t = 5$, feature f_8 emerges, features f_5, f_6 and f_7 disappear, and previously disappeared feature f_2 reoccurs. Class c_1 disappears from the label space.

However, existing algorithms that handle class evolution and feature evolution meanwhile, are limited to processing the data stream in a chunk-by-chunk manner [3], [9]. This approach necessitates large amounts of data storage for model adaptation and hinders the model from promptly capturing and adapting to changes in the data stream. Subsequent studies are concentrated on building online learning algorithms to address either class evolution or feature evolution, not both.

Received 22 September 2025; revised 2 June 2026; accepted 26 June 2026. Date of publication 29 June 2026; date of current version 8 August 2026. This work was supported in part by Research and Application on Intelligent Highway Inspection, Diagnosis, and Maintenance Decision-Making under Grant K240401HF and in part by the internal grants of Lingnan University. Recommended for acceptance by X. Liu. *(Corresponding author: Xin Yao.)*

Zhi Cao is with the Anhui Transport Consulting & Design Institute Company, Ltd, Hefei 230088, China, also with the University of Science and Technology of China, Hefei 230026, China, and also with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: caoz@mail.sustech.edu.cn).

Peijia Qin is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: qinpj2021@mail.sustech.edu.cn).

Chin-Teng Lin is with the CIBCI lab, Centre for Artificial Intelligence, the School of Computer Science, University of Technology, Sydney, NSW 2007, Australia (e-mail: chin-teng.lin@uts.edu.au).

Xin Yao is with the School of Data Science, Lingnan University, Hong Kong, SAR, China (e-mail: xinyao@ln.edu.hk).

Digital Object Identifier 10.1109/TKDE.2026.3708299

In online learning with feature evolution, a vast group of algorithms have been proposed [10], covering application scenarios from supervised [11] to semi-supervised learning [12] and accommodating data streams from the trapezoidal case [13] where the number of features increases monotonously, to the general capricious case [14] where the feature space varies randomly. However, all of these methods inherently assume that the set of classes is fixed throughout the entire data stream.

In online learning with class evolution, class-based ensemble models using one-versus-all (OVA) classifiers [15] have shown significant progress in addressing various forms of class evolution [1], effectively accommodating a broader spectrum of changes in class distributions [2]. However, these models inherently operate under the assumption of a fixed feature space.

To the best of our knowledge, no existing work is capable of addressing both feature evolution and class evolution simultaneously in an online learning context. We begin our investigations of this problem by constructing several baseline models based on the state-of-the-art approaches in either feature evolution or class evolution. Specifically, the ensemble of ensemble OVA framework [2] for class evolution, and the Passive-Aggressive (PA) model [16] for feature evolution. The constructed baseline ensembles of PA models, where PA models are configured to function as OVA classifiers, appear to provide viable solutions. We then go beyond merely integrating state-of-the-art approaches by analyzing their limitations. Although PA methods are ubiquitous in solving online learning with feature evolution [10], the accommodation of the bias term to the dynamic data space is generally ignored in previous work. In addition, previous online learning algorithms for handling class evolution primarily address the imbalance between the positive and negative classes in each OVA classifier [1], [2]. However, the negative class is not a singular entity but consists of multiple distinct classes. The imbalance issue among these classes was overlooked.

In this paper, we propose a novel Ensemble of Passive-Aggressive Model (EPAM) to address the problem of online learning in the open data space. This model manages both feature evolution and class evolution using only a single available instance each time, eliminating the need to store any historical data. In EPAM, a novel feature contributed bias classifier (FCBC) serves as the base OVA classifier, with its bias term adapting dynamically to diverse feature spaces formed by varying feature combinations in the open data space, therefore, delivering a more accurate decision boundary. Furthermore, a novel model adaptation strategy is designed to balance error feedback among classes that belong to the negative class of each FCBC, enhancing the multi-class classification performance of EPAM in the open data space as the set of classes evolve. Comprehensive experimental studies on both synthetic and real-world data streams demonstrate the superiority of EPAM, and highlight the significance of the proposed components in managing data streams in the open data space.

In summary, our main contributions are as follows:

- We propose a novel model EPAM to address the problem of online learning in the open data space, where both feature evolution and class evolution may occur.
- We construct several baseline models grounded in state-of-the-art online learning algorithms for either class evolution or feature evolution, and analyze their limitations in handling real-world data streams with both class and feature evolution.
- To address limitations of existing methods, in EPAM, we integrate a novel FCBC model with an adaptive bias term to handle feature evolution, and a novel model adaptation strategy to balance error feedback among classes that belong to the negative class.
- We empirically demonstrate the superior performance of the proposed model EPAM and the significance of each proposed component on diverse synthetic and real-world data streams.

II. Related Work

A. Feature and Class Evolution

In the open data space, data streams are inherently unpredictable, with both the label space and feature space undergoing continuous evolution. Masud et al. first investigated these issues and propose ensemble models DXMiner [9] and MCM [3]. In these models, the outdated base classifier is supplanted by a newly trained base classifier on the latest data chunk, thereby adapting the model to class evolution. The feature space conversion method is introduced to establish a homogeneous feature space in each data chunk, enabling the training of the new base classifier amidst feature evolution. However, these models process the data stream chunk-by-chunk, constructing a new base classifier for each chunk to update the ensemble model. In scenarios of online learning, only the current instance is available. It is impractical to build a new base classifier for each input instance.

Subsequent researchers focus on developing online learning algorithms that address data streams characterized by issues of either feature evolution [10] or class evolution [1].

B. Online Learning With Feature Evolution

In online learning with feature evolution, a variety of scenarios have been investigated. The trapezoidal scenario, which is characterized by the monotonously increasing number of features, is first handled by STSD [17] and OL$_{SF}$ [13]. These algorithms are constructed based on the PA model [18]. The weights of novel features are initialized in the linear model, which is then updated to maximize the margins in the current feature space. The nonlinear separable case is handled in SLFN-S [19] through the construction of an adaptive neural network with shortcut connections. Further investigations have extended to active learning [20], bandit feedback settings [21], and semi-supervised outlier detection problem [22] in trapezoidal data streams.

Hou et al. [23] consider the disappearance of features from the previous round, noting the existence of a brief overlapping period during which disappearing features coincide with novel features. In their model FESL [23], linear correlation between old and new features is established to adapt the classifier on new features. Algorithms such as EDM [24] and FEMC [25] are

proposed to address the concept drift issue [26]. PUFE [27] is proposed to handle the scenario where the lengths of the overlap periods vary randomly for different disappearing features.

The challenge presented by high-dimensional feature spaces that involve complex feature interplay is further addressed by OLD^{3}S [28]. In contrast to the previous scenarios that rely on specific assumptions regarding the dynamics of feature evolution, Beyazit et al. [14] and He et al. [11] explore a more general scenario without any assumption, termed capricious data streams, where the feature space varies arbitrarily. OLVF [14] introduces the projection confidence to estimate the probability that the classifier makes a correct prediction in the current feature space. In OCDS [11] and GLSC [29], generative graphical models are incorporated to reconstruct every input instance in the universal feature space composed of all observed features. A similar approach is also integrated in AGDES [12] to make the distance measurement feasible in a varying feature space, handling the semi-supervised capricious data streams. Semi-supervised trapezoidal and capricious data streams are further investigated in OL-MDISF [30], which studies online learning in the presence of mixed feature types and drifted data distributions.

More recently, OLI^{2}DS [16], which is constructed based on the PA model [18], achieves state-of-the-art performance across various scenarios involving different dynamics of feature evolution. A dynamic cost strategy is implemented to handle class imbalance and the concept of informativeness is employed to assess features, assigning higher update weights to those with higher informativeness. The latter work OLIFL [31] inherits the idea of informativeness measurement and extends the model to semi-supervised scenarios. Although a broad range of feature evolution scenarios has been explored, existing research generally assumes that the set of classes is fixed. Consequently, these methods are incapable of addressing data streams with class evolution.

C. Online Learning With Class Evolution

Although models have been proposed to adapt classifiers to a changing set of classes, a significant number of these models require substantial storage of historical data [33]. In online learning with class evolution, algorithms are required to maintain an effective classification model with a single available data instance [2]. In [32], the adapted OVA decision trees are proposed, initializing a new subtree upon the arrival of an instance of a novel class. The sub-tree corresponding to the arriving instance is updated, along with the sub-tree that is most likely to misclassify this instance. In CBCE [1], a one-sided undersampling method is incorporated to address the dynamic class imbalance arising from the gradual evolution of classes, and an innovative activate/deactivate strategy is designed to adapt OVA classifiers to class evolution. In the latest research, EEOF [2], characterized by its ensemble of ensemble OVA framework, achieves the state-of-the-art classification performance in online learning with class evolution. In EEOF, a novel cost-sensitive model adaptation method and the confidence-triggered fallback mode are designed to tackle a broader spectrum of class evolution scenarios. However, all these online learning algorithms implicitly assume that the feature space remains fixed, and consequently only address the issue of class evolution.

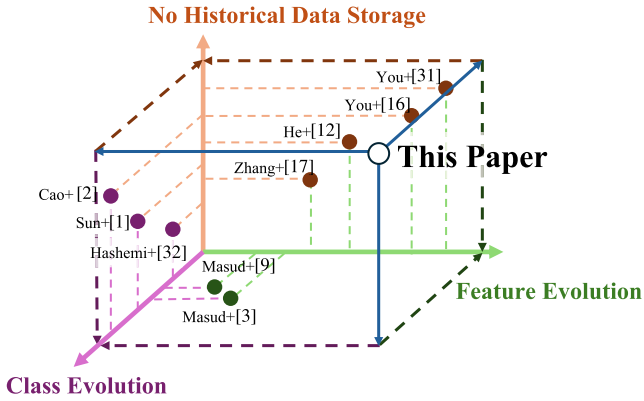


Fig. 2. The projected positions of the papers along the axes are utilized solely to illustrate the relative order of their publication years. In this paper, we introduce a novel model, EPAM, that is capable of handling class evolution and feature evolution with no historical data storage. Papers included in this figure are Masud et al. [3], [9], Hashemi et al. [32], Zhang et al. [17], He et al. [12], You et al. [16], [31], Sun et al. [1], and Cao et al. [2].

In summary, a more general online learning algorithm, one that imposes no fixed-set assumptions on either the feature space or the label space, is needed, as shown in Fig. 2.

III. Problem Formulation

In this section, we present a formal formulation of online learning in the open data space. Given a continuous data stream $\{(\boldsymbol{x}_t, y_t)\}_1^{+\infty}$, where $\boldsymbol{x}_t \in \mathcal{X}_t \subseteq \mathbb{R}^{d_t}$ is a feature vector of length d_t, and $y_t \in \mathcal{Y}_t$ is the associated class label. Here, $\mathcal{X}_t$ denotes the feature space at time t with $\mathcal{X}_t = \{f_i | \text{feature } f_i \text{ occurred at time } t\}$ and $|\mathcal{X}_t| = d_t$. $\mathcal{Y}_t$ denotes the label space at time t. Each element in $\mathcal{Y}_t$ is the class c_i with a positive prior probability at time t, i.e. $\mathcal{Y}_t = \{c_i | p_t^{(i)} > 0\}$. In online learning, only one instance $\boldsymbol{x}_t$ is available at time t. The true label y_t of $\boldsymbol{x}_t$ arrives after the prediction $\hat{y}_t$ is made before time $t+1$.

In the open data space, both $\mathcal{X}_t$ and $\mathcal{Y}_t$ change over time, referred to as feature evolution and class evolution respectively. Taking class evolution as an example, based on how classes evolve, class evolution is categorized into three principle forms: 1) class emergence, 2) class disappearance, and 3) class reoccurrence. A class c_{novel} emerges at time t if $c_{novel} \in \mathcal{Y}_t$ and $c_{novel} \notin \bigcup_{i=1}^{t-1} \mathcal{Y}_i$. A class c_{disp} disappears at time t if $c_{disp} \notin \mathcal{Y}_t$ and $c_{disp} \in \mathcal{Y}_{t-1}$. A class $c_{reoccur}$ disappears at time d and reoccurs at time t if $c_{reoccur} \in \mathcal{Y}_t$ and $c_{reoccur} \notin \bigcup_{i=d}^{t-1} \mathcal{Y}_i$ and $c_{reoccur} \in \mathcal{Y}_{d-1}$, where $d < t$. The forms of feature evolution can be categorized in a similar way.

The task of online learning in the open data space is to construct an online classifier $\mathcal{F} : \mathcal{X}_t \to \mathcal{Y}_t$ that is capable of 1) making accurate prediction $\hat{y}_t$ for arriving instance $\boldsymbol{x}_t$ at time t, and 2) updating the classifier after receiving the true label y_t.

IV. Baseline Ensemble of PA Models

In this section, we first provide a brief overview of the general formulations in current research addressing class evolution and

feature evolution. We then construct baseline models for online learning in the open data space based on state-of-the-art online learning algorithms in class evolution and feature evolution. Finally, we analyze the limitations of existing approaches in managing data streams in the open data space.

A. Preliminary

The prevailing approach of existing online learning algorithms in handling class evolution involves maintaining an ensemble of one-versus-all (OVA) classifiers [34], [35], which transforms the n-class classification into n binary classifications. Each OVA classifier $\mathcal{F}_i$ in the ensemble predicts whether an incoming instance $\boldsymbol{x}_t$ belongs to class c_i (designated as the positive class $+1$ for $\mathcal{F}_i$) or not (designated as the negative class -1 for $\mathcal{F}_i$). The final prediction is classified to the class with the highest prediction score:

$$\hat{y}_t = \arg\max_i \mathcal{F}_i(\boldsymbol{x}_t). \tag{1}$$

A new OVA classifier is initialized and added to the ensemble when a novel class emerges. The OVA classifier corresponding to any disappeared class is excluded from the classification process in (1). When a previously disappeared class reoccurs, its corresponding OVA classifier is reintroduced for classification [1].

The prevalent approaches of existing online learning algorithms in solving feature evolution are Passive-Aggressive (PA) models. Here, we present a general formulation of PA models as outlined in [10], using our notations. In PA models, each component of the classifier's weight vector corresponds to an observed feature. At time t, the instance $\boldsymbol{x}_t \in \mathcal{X}_t$ arrives. The weight vector $\boldsymbol{w}_t$ is associated with the set of observed feature before t, i.e. $\boldsymbol{w}_t \in \bigcup_{i=1}^{t-1} \mathcal{X}_i \subseteq \mathbb{R}^{u_{t-1}}$, where u_{t-1} represents the number of all features observed up to time $t-1$. Based on the feature space of $\boldsymbol{w}_t$, the feature space of $\boldsymbol{x}_t$ can be split into three parts: common, disappeared, and novel feature spaces. In the common feature space, features are both observed previously and in $\boldsymbol{x}_t$, i.e. $f_c \in \mathcal{X}_t \cap (\bigcup_{i=1}^{t-1} \mathcal{X}_i)$. In the disappeared feature space, features are observed previously and absent in $\boldsymbol{x}_t$, i.e. $f_d \in (\bigcup_{i=1}^{t-1} \mathcal{X}_i) \setminus \mathcal{X}_t$. In the novel feature space, features are all novel ones, i.e. $f_n \in \mathcal{X}_t \setminus (\bigcup_{i=1}^{t-1} \mathcal{X}_i)$.

Let the superscripts "c", "d" and "n" denote the projection of a vector onto the common, disappeared, and novel feature spaces, respectively. In the prediction phase, the decision is made with the projections of $\boldsymbol{w}_t$ and $\boldsymbol{x}_t$ on the common feature space, as defined as:

$$\hat{y}_t = \text{sign}\left(\boldsymbol{w}_t^c \cdot \boldsymbol{x}_t^c\right). \tag{2}$$

The bias term is omitted in (2), which is either set to 0 or incorporated by introducing a dummy input attribute $x_{t,0} = 1$ into $\boldsymbol{x}_t$. In the update phase, the weight vector is updated through minimization of the following objective function over the variable vector $\boldsymbol{w}$ [10]:

$$\begin{aligned} \boldsymbol{w}_{t+1} = & \arg\min_{\boldsymbol{w}=[\boldsymbol{w}^d,\boldsymbol{w}^c,\boldsymbol{w}^n]} d(\boldsymbol{w},\boldsymbol{w}_t) + \gamma||\boldsymbol{w}^n||_p + C\xi \\ \text{s.t.} \quad & l(\boldsymbol{w};(\boldsymbol{x}_t,y_t)) \le \xi, \xi \ge 0, \end{aligned} \tag{3}$$

where $\boldsymbol{w}^d$, $\boldsymbol{w}^c$, and $\boldsymbol{w}^n$ denote the projection of the variable vector $\boldsymbol{w}$ onto the common, disappeared, and novel feature space respectively. $d(\cdot,\cdot)$ is the distance function to measure the update of the weight vector on previously seen features. The l_p-norm serves as a regularization term for the weight vector in the novel feature space. ξ is a nonnegative slack variable that facilitates a soft-margin boundary [36]. $l(\boldsymbol{w};(\boldsymbol{x}_t,y_t))$ represents the loss function of a classifier with the weight vector $\boldsymbol{w}$ when making the prediction on the instance $(\boldsymbol{x}_t,y_t)$. γ and C are trade-off parameters to control the influence of different terms in the objective function. Generally, the hinge loss is the default choice for the loss function.

B. Baseline Models

Note that PA models are binary classifiers. To develop online learning algorithms that manage data streams in the open data space, a straight forward approach is to assemble PA models as base OVA classifiers. Following the state-of-the-art method in online learning with class evolution, EEOF [2], we adapt different PA models within the ensemble of ensemble framework, thereby constructing a group of baseline ensembles of PA models. In each baseline model, an ensemble of PA models is maintained as the OVA classifiers for each class, namely, $\mathcal{F}_i = \{\mathcal{F}_{i1}, \mathcal{F}_{i2}, \ldots, \mathcal{F}_{im}\}$. Here, m is the ensemble size. The prediction score for class c_i is the average prediction score across all PA models in $\mathcal{F}_i$, i.e. $\mathcal{F}_i(\boldsymbol{x}_t) = \frac{1}{m}\sum_{j=1}^{m} \mathcal{F}_{ij}(\boldsymbol{x}_t)$. The final prediction result $\hat{y}_t$ is determined as:

$$\hat{y}_t = \arg\max_i \frac{1}{m}\sum_{j=1}^{m} \mathcal{F}_{ij}(\boldsymbol{x}_t), \mathcal{F}_{ij}(\boldsymbol{x}_t) = \boldsymbol{w}_{ij,t}^c \cdot \boldsymbol{x}_t^c, \tag{4}$$

where $\boldsymbol{w}_{ij,t}^c$ represents the projection of the weight vector $\boldsymbol{w}_{ij,t}$ for the j-th PA model corresponding to class c_i in the common feature space. In addition to the strategy of excluding disappeared classes and re-including reoccurred classes in the prediction process [1], the confidence-triggered fallback mode is also integrated, ensuring that disappeared classes are re-included when PA models for existing classes fail to provide reliable predictions. Details of the confidence-triggered fallback mode can be found in [2].

Class evolution leads to the dynamic changes of prior probabilities of classes. For the PA model $\mathcal{F}_{ij}$ in a n-class classification problem, class c_i is considered as the positive class while the remaining $n-1$ classes are considered as the negative class. For the adaptation of $\mathcal{F}_{ij}$, the assigned binary label $y_{i,t}$ is used in place of the true label y_t, defined as:

$$y_{i,t} = \begin{cases} +1, & y_t = c_i \\ -1, & y_t \ne c_i. \end{cases} \tag{5}$$

The loss weight parameter $r_{ij,t}$ is introduced to modulate the loss on the arriving instance $(\boldsymbol{x}_t, y_{i,t})$, thereby balancing the error feedback between the positive and negative classes for $\mathcal{F}_{ij}$. After integrating the PA model into the ensemble of ensemble framework and taking the loss weight into consideration, the objective of the PA model $\mathcal{F}_{ij}$ is:

$$\begin{aligned} \boldsymbol{w}_{ij,t+1} = & \arg\min_{\boldsymbol{w}=[\boldsymbol{w}^d,\boldsymbol{w}^c,\boldsymbol{w}^n]} d(\boldsymbol{w},\boldsymbol{w}_{ij,t}) + \gamma||\boldsymbol{w}^n||_p + C\xi \\ \text{s.t.} \quad & r_{ij,t} l(\boldsymbol{w};(\boldsymbol{x}_t,y_{i,t})) \le \xi, \xi \ge 0, \end{aligned} \tag{6}$$

where $\boldsymbol{w}$ is the optimization variable vector. The loss function is then defined as:

$$l(\boldsymbol{w};(\boldsymbol{x}_t,y_{i,t}))=\max\{0,1-y_{i,t}(\boldsymbol{w}_{\|\boldsymbol{x}_t}\cdot\boldsymbol{x}_t)\}, \tag{7}$$

where $\boldsymbol{w}_{\|\boldsymbol{x}_t}$ represents the projection of the variable vector $\boldsymbol{w}$ onto the feature space of $\boldsymbol{x}_t$.

In EEOF [2], the prior probability $p_t^{(i)}$ of each class c_i at time t is estimated using the time decayed method [37], [38]:

$$\hat{p}_t^{(i)}=\beta\hat{p}_{t-1}^{(i)}+(1-\beta)\mathbb{1}[y_t==c_i], \tag{8}$$

where $\hat{p}_t^{(i)}$ is the estimated prior probability and β is the time decay factor. The estimated prior probability $\hat{p}_t^{(i)}$ is then utilized to balance the error feedback between positive and negative classes. Specifically, the loss weight $r_{ij,t}$ for the PA model $\mathcal{F}_{ij}$ is generated in the form:

$$r_{ij,t}\sim\begin{cases}Poisson(1), & y_t=c_i\\ Poisson(\hat{p}_t^{(i)}/\left(1-\hat{p}_t^{(i)}\right)), & y_t\neq c_i.\end{cases} \tag{9}$$

The state-of-the-art method in online learning with feature evolution is OLI^{2}DS [16], which is developed based on the PA model and employs the cumulative class ratio to address the issue of class imbalance in the binary classification problem. Furthermore, it employs the concept of informativeness to assign varying update weights to different features. OL$_{SF}$ [13] is the benchmark PA model in online learning with feature evolution. By plugging implementations of these PA models into the objective function (6), we can construct several baseline models. Specifically, these baseline ensembles of PA models are:

- *EEOF-OLI^{2}DS$_1$:* Plugging OLI^{2}DS into the objective function (6) with the loss weight $r_{ij,t}$ to be a multiple of the cumulative class ratio, as defined in the OLI^{2}DS [16] model:

$$r_{ij,t}=\begin{cases}\theta*n_{ij,t}^{-}/\left(n_{ij,t}^{+}+n_{ij,t}^{-}\right), & y_t=c_i\\ \theta*n_{ij,t}^{+}/\left(n_{ij,t}^{+}+n_{ij,t}^{-}\right), & y_t\neq c_i,\end{cases} \tag{10}$$

  where θ is the scaling factor, $n_{ij,t}^{+}$ and $n_{ij,t}^{-}$ are counts of instances from the positive and negative classes of $\mathcal{F}_{ij}$ until time t, respectively.
- *EEOF-OLI^{2}DS$_2$:* Same as EEOF-OLI^{2}DS$_1$ with the *exception* that the loss weight $r_{ij,t}$ is determined by the time-decayed dynamic class ratio in EEOF [2], as described in (8), (9).
- *EEOF-OL$_{SF}$:* Plugging OL$_{SF}$ into the objective function (6), with the loss weight $r_{ij,t}$ determined by the time-decayed dynamic class ratio in EEOF [2], as described in (8), (9).

C. Limitation Analysis

As presented above, we introduce a construction strategy that integrates state-of-the-art online learning algorithms for either class evolution or feature evolution to handle online learning in the open data space. However, several limitations still remain when addressing real-world data streams. In current PA models [4], [10], [13], [14], [16], [17], [19], [21], [31], as seen in (2), the bias term is excluded from feature evolution, either by incorporating it into the weight vector by inventing a dummy input attribute $x_{t,0}=1$ or simply assuming it to be 0. In other words, the bias term in these models is the same value or simply 0 for diverse feature spaces formed by different possible combinations of features. Consider a binary classification data stream in which classes are linearly separable with every single feature. In this scenario, we cannot guarantee that a zero or a fixed bias is appropriate for any linear model on any single feature. Moreover, even if an optimal bias can be identified for each linear model on individual features, the found bias is still unlikely to be perfectly suitable for a linear model operating in a feature space formed of any combination of these features. In the real-world open data space, the number of possible feature spaces is uncountable and the feature evolution is unpredictable. The bias term in the PA model is required to adapt to the varying feature spaces for constructing more accurate decision boundary.

Current online learning algorithms for class evolution [1], [2] primarily focus on the imbalance between the positive and negative classes in each OVA classifier $\mathcal{F}_i$. As described in (9), EEOF, the state-of-the-art online learning algorithm for class evolution, generates loss weights using the ratio of the estimated prior probability of the positive class $\hat{p}_t^{(i)}$ to that of the negative class $1-\hat{p}_t^{(i)}$, applied uniformly to all instances with $y_t\neq c_i$. However, the negative class is not a singular entity but encompasses multiple classes. Previous work applies the same sampling ratio [1] or loss weight [2] to all instances from different classes designated as the negative class, potentially assuming that classes belonging to the negative class are evenly distributed, which is overly idealistic in real-world data streams with class evolution. For example, for a class c_k with a considerably small $p_t^{(k)}$, instances in this class are designated as negative for $\mathcal{F}_i$ with $i\neq k$. Generally, the number of instances in the positive class is smaller than that in the negative class (i.e., the remaining classes). The error feedback from c_k for $\mathcal{F}_i$ diminishes with the other classes designated as the negative class due to the balancing techniques in the existing work. However, the error feedback from the small class c_k should be overweighted. Focusing solely on balancing the positive and negative class and ideally treating the negative class as a single entity may degrade the overall multi-class classification performance.

V. The Proposed Model: EPAM

In light of aforementioned limitations, we propose EPAM to tackle the problem of online learning in the open data space. In this section, we will introduce the details of EPAM.

A. Feature Contributed Bias Classifier (FCBC)

As analyzed in the previous section, the bias term is required to be capable of accommodating different feature spaces in the data stream with feature evolution. We adopt the "voting weight" perspective introduced in prior work [10] to motivate our proposed model. Under this view, each feature f_i with value x_i contributes to the final prediction through an associated "voting weight" w_i in a linear classifier $\hat{y}_t=\text{sign}(\boldsymbol{w}_t^T x_t)$. Let

the class-conditional distributions of feature x_i be $P(x_i \mid y = 1)$ and $P(x_i \mid y = -1)$. The voting neutrality point x_i^* is defined as the value at which the posterior probabilities of the two classes are equal, i.e.:

$$P(y = 1 \mid x_i = x_i^*) = P(y = -1 \mid x_i = x_i^*) = \frac{1}{2}. \quad (11)$$

Accordingly, the "voting value" of the i-th feature to the classifier can be expressed as $w_i(x_{t,i} - x_i^*) = w_i x_{t,i} - w_i x_i^*$. Notably, the term $w_i x_i^*$ is intrinsically associated with the i-th feature. A scalar bias term is insufficient to account for the presence or absence of individual features. When feature f_i is absent in a new feature space, a scalar bias fails to account for the contribution of the missing feature, introducing a systematic error of $-w_i x_i^*$. In practice, assuming $x_i^* = 0$ is generally unrealistic. Even if features are standardized to have zero mean, the neutrality point x_i^* does not necessarily coincide with zero. Therefore, to properly adapt to the evolving feature space, the bias term should be designed to explicitly account for the presence or absence of individual features. Let $b_i = -w_i x_i^*$, the classifier can then be formulated as:

$$\hat{y}_t = \text{sign}\left(\boldsymbol{w}_t^T \boldsymbol{x}_t + \boldsymbol{b}_t^T \mathbf{1}_t\right), \quad (12)$$

where the i-th value of $\mathbf{1}_t$ is 1 if f_i is present at time t, and 0 otherwise.

In EPAM, the feature contributed bias classifier (FCBC) is designed as the base OVA classifier $\mathcal{F}_{ij}$ with an adaptive bias vector $\boldsymbol{b}_{ij,t}$ capable of adapting to diverse feature spaces, offering more accurate decision boundary for classification. For every observed feature, there is a corresponding value in $\boldsymbol{b}_{ij,t}$. Therefore, $\boldsymbol{b}_{ij,t}$ has the same shape as the weight vector $\boldsymbol{w}_{ij,t}$, i.e. $\boldsymbol{w}_{ij,t}, \boldsymbol{b}_{ij,t} \in \bigcup_{i=1}^{t-1} \mathcal{X}_i \subseteq \mathbb{R}^{u_{t-1}}$, where u_{t-1} represents the number of all features observed up to time $t-1$. For an arriving instance $\boldsymbol{x}_t$, $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ can be divided into projections on the common and the disappeared feature spaces, i.e. $\boldsymbol{w}_{ij,t} = [\boldsymbol{w}_{ij,t}^d, \boldsymbol{w}_{ij,t}^c]$ and $\boldsymbol{b}_{ij,t} = [\boldsymbol{b}_{ij,t}^d, \boldsymbol{b}_{ij,t}^c]$. In the prediction phase, the prediction of $\mathcal{F}_{ij}$ is calculated on the common feature space:

$$\mathcal{F}_{ij}(\boldsymbol{x}_t) = \boldsymbol{w}_{ij,t}^c \cdot \boldsymbol{x}_t^c + \boldsymbol{b}_{ij,t}^c \cdot \mathbf{1}^c, \quad (13)$$

where $\mathbf{1}^c$ is the vector filled with ones in the common feature space. For comparison, the formulation with a dummy feature in previous work [4], [10], [13], [14], [16], [17], [19], [21], [31] is:

$$\mathcal{F}_{ij}(\boldsymbol{x}_t) = \boldsymbol{w}_{ij,t}^c \cdot \boldsymbol{x}_t^c + b \cdot x_{t,0}, \quad (14)$$

where $x_{t,0} = 1$ is the dummy feature and b is the bias term. Equations (13) and (14) are equivalent when the common feature space is fixed across all time t, where no feature evolution occurs in the data stream and $\boldsymbol{b}_{ij,t}^c \cdot \mathbf{1}^c$ reduces to a constant global bias b. However, as the feature space evolves over time, (13) and (14) become non-equivalent. In FCBC, only biases corresponding to features that are both present in the current instance $\boldsymbol{x}_t$ and available in the bias vector $\boldsymbol{b}_{ij,t}$ are used for prediction. This strategy explicitly accounts for the influence of the evolving feature space on the bias term, allowing the bias to dynamically align with the changing feature space rather than remaining fixed as a global constant, thereby enhancing the model's robustness in handling feature evolution.

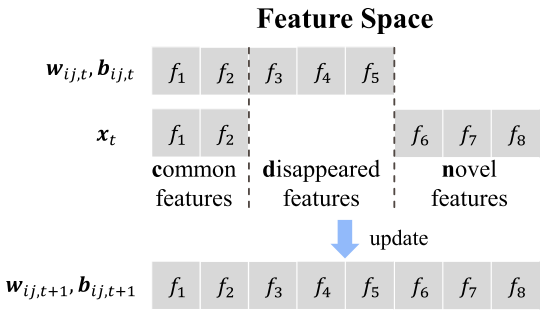


Fig. 3. Before the update, $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ contains values only on the common feature space $\{f_1, f_2\}$ and the disappeared feature space $\{f_3, f_4, f_5\}$ compared with the feature space of $\boldsymbol{x}_t$. After the update, $\boldsymbol{w}_{ij,t+1}$ and $\boldsymbol{b}_{ij,t+1}$ are extended to include values on the novel feature space $\{f_6, f_7, f_8\}$.

In the update phase, the weight vector and the bias vector are both extended to novel feature space. By employing the square of the euclidean distance as $d(\cdot,\cdot)$ along with $\gamma = 1$ into the objective function (6) and integrating the update of the bias vector, the objective function of FCBC $\mathcal{F}_{ij}$ over the variable vectors $\boldsymbol{w}$ and $\boldsymbol{b}$ is then designed in the following form:

$$\begin{aligned}\boldsymbol{w}_{ij,t+1}, \boldsymbol{b}_{ij,t+1} = \underset{\substack{\boldsymbol{w}=[\boldsymbol{w}^d,\boldsymbol{w}^c,\boldsymbol{w}^n] \\ \boldsymbol{b}=[\boldsymbol{b}^d,\boldsymbol{b}^c,\boldsymbol{b}^n]}}{\arg\min} & \frac{1}{2}(\|\boldsymbol{w}^d - \boldsymbol{w}_{ij,t}^d\|^2 \\ & + \|\boldsymbol{w}^c - \boldsymbol{w}_{ij,t}^c\|^2 + \|\boldsymbol{w}^n\|^2) + \frac{A}{2}(\|\boldsymbol{b}^d - \boldsymbol{b}_{ij,t}^d\|^2 \\ & + \|\boldsymbol{b}^c - \boldsymbol{b}_{ij,t}^c\|^2 + \|\boldsymbol{b}^n\|^2) + C\xi \\ \text{s.t.} \quad & r_{ij,t} l_{i,t}(\boldsymbol{w}^c, \boldsymbol{w}^n, \boldsymbol{b}^c, \boldsymbol{b}^n) \le \xi, \xi \ge 0, \end{aligned} \quad (15)$$

where $r_{ij,t}$ is the loss weight, ξ is the slack variable, $A > 0$ adjusts the update trade-off between the weight vector and the bias vector, $C > 0$ controls the trade-off between rigidness and slackness, and the loss function is defined as:

$$\begin{aligned} l_{i,t}(\boldsymbol{w}^c, \boldsymbol{w}^n, \boldsymbol{b}^c, \boldsymbol{b}^n) &= l(\boldsymbol{w}, \boldsymbol{b}; (\boldsymbol{x}_t, y_{i,t})) \\ &= \max\{0, 1 - y_{i,t}(\boldsymbol{w}^c \cdot \boldsymbol{x}_t^c + \boldsymbol{w}^n \cdot \boldsymbol{x}_t^n \\ &\quad + \boldsymbol{b}^c \cdot \mathbf{1}^c + \boldsymbol{b}^n \cdot \mathbf{1}^n)\}, \end{aligned} \quad (16)$$

where $y_{i,t}$ is the designated label defined as in (5).

The objective function (15) of FCBC aims to achieve two goals: 1) to retain the information learned previously by minimizing the differences between the current and previous classifiers in terms of the weight and bias vectors in the disappeared and common feature spaces, and 2) to extend the classifier to the novel feature space (an example illustrated in Fig. 3) by minimizing its corresponding regularization terms, i.e. $\|\boldsymbol{w}^n\|^2$ and $\|\boldsymbol{b}^n\|^2$. To solve the objective function (15), we employ the Lagrangian function and the Karush-Kuhn-Tucker (KKT) conditions [39], and obtain:

$$\begin{aligned} \mathcal{L}(\boldsymbol{w}, \boldsymbol{b}, \xi, \tau, \eta) = & \frac{1}{2}(\|\boldsymbol{w}^d - \boldsymbol{w}_{ij,t}^d\|^2 + \|\boldsymbol{w}^c - \boldsymbol{w}_{ij,t}^c\|^2 \\ & + \|\boldsymbol{w}^n\|^2) + \frac{A}{2}(\|\boldsymbol{b}^d - \boldsymbol{b}_{ij,t}^d\|^2 + \|\boldsymbol{b}^c - \boldsymbol{b}_{ij,t}^c\|^2 + \|\boldsymbol{b}^n\|^2) \\ & + C\xi + \tau(r_{ij,t} l_{i,t}(\boldsymbol{w}^c, \boldsymbol{w}^n, \boldsymbol{b}^c, \boldsymbol{b}^n) - \xi) - \eta\xi \end{aligned} \quad (17)$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}^d} = 0 \implies \boldsymbol{w}^d = \boldsymbol{w}^d_{ij,t} \tag{18}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}^c} = 0 \implies \boldsymbol{w}^c = \boldsymbol{w}^c_{ij,t} + \tau r_{ij,t} y_{i,t} \boldsymbol{x}^c_t \tag{19}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}^n} = 0 \implies \boldsymbol{w}^n = \tau r_{ij,t} y_{i,t} \boldsymbol{x}^n_t \tag{20}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^d} = 0 \implies \boldsymbol{b}^d = \boldsymbol{b}^d_{ij,t} \tag{21}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^c} = 0 \implies \boldsymbol{b}^c = \boldsymbol{b}^c_{ij,t} + \frac{\tau r_{ij,t} y_{i,t}}{A} \mathbf{1}^c \tag{22}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^n} = 0 \implies \boldsymbol{b}^n = \frac{\tau r_{ij,t} y_{i,t}}{A} \mathbf{1}^n \tag{23}$$

$$\frac{\partial \mathcal{L}}{\partial \xi} = 0 \implies \eta = C - \tau, \tag{24}$$

where τ and η are Lagrange multipliers, and $\mathbf{1}^n$ is the vector filled with ones in the novel feature space. Plugging the (18), (19), (20), (21), (22), (23), (24) into (17) and setting the partial derivative of $\mathcal{L}(\tau)$ with respect to τ to zero, we obtain:

$$\begin{aligned}\mathcal{L}(\tau) = & -\frac{\tau^2 r^2}{2}\left(\|\boldsymbol{x}^c_t\|^2 + \|\boldsymbol{x}^n_t\|^2 + \frac{\|\mathbf{1}^c\|^2 + \|\mathbf{1}^n\|^2}{A}\right) \\ & + \tau r_{ij,t} l_{i,t}(\boldsymbol{w}^c_{ij,t}, \boldsymbol{w}^n_{ij,t}, \boldsymbol{b}^c_{ij,t}, \boldsymbol{b}^n_{ij,t})\end{aligned} \tag{25}$$

$$\tau_{ij,t} = \min\left\{C, \frac{l_{i,t}(\boldsymbol{w}^c_{ij,t}, \boldsymbol{w}^n_{ij,t}, \boldsymbol{b}^c_{ij,t}, \boldsymbol{b}^n_{ij,t})}{r_{ij,t}(\|\boldsymbol{x}_t\|^2 + d_t/A)}\right\}, \tag{26}$$

where d_t is the number of features occurred at time t, i.e. $d_t = |\mathcal{X}_t|$, and $\boldsymbol{w}^n_{ij,t}, \boldsymbol{b}^n_{ij,t} = \mathbf{0}^n$. Then, the update strategy of FCBC is:

$$\boldsymbol{w}_{ij,t+1} = \left[\boldsymbol{w}^d_{ij,t}, \boldsymbol{w}^c_{ij,t} + \tau_{ij,t} r_{ij,t} y_{i,t} \boldsymbol{x}^c_t, \tau_{ij,t} r_{ij,t} y_{i,t} \boldsymbol{x}^n_t\right] \tag{27}$$

$$\boldsymbol{b}_{ij,t+1} = \left[\boldsymbol{b}^d_{ij,t}, \boldsymbol{b}^c_{ij,t} + \frac{\tau_{ij,t} r_{ij,t} y_{i,t} \mathbf{1}^c}{A}, \frac{\tau_{ij,t} r_{ij,t} y_{i,t} \mathbf{1}^n}{A}\right]. \tag{28}$$

Similar to previous studies of PA models [13], [16], [17], we introduce the sparsity strategy, offering an alternative for FCBC managing high-dimensional data streams. In the sparsity strategy, a proportion $B \in (0, 1]$ of the parameters in $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ with the largest absolute values is retained and the remaining $(1 - B)$ proportion is truncated to 0. To mitigate the impact of truncation, $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ are first projected onto a L_1 ball [40], thereby ensuring that the removal of the elements with smallest absolute values results in small changes to the weight and bias vectors. The projections are defined as:

$$\boldsymbol{w}_{ij,t+1} = \min\left\{1, \frac{\lambda}{\|\boldsymbol{w}_{ij,t+1}\|_1 + \|\boldsymbol{b}_{ij,t+1}\|_1}\right\} \boldsymbol{w}_{ij,t+1} \tag{29}$$

$$\boldsymbol{b}_{ij,t+1} = \min\left\{1, \frac{\lambda}{\|\boldsymbol{w}_{ij,t+1}\|_1 + \|\boldsymbol{b}_{ij,t+1}\|_1}\right\} \boldsymbol{b}_{ij,t+1}, \tag{30}$$

where $\lambda > 0$ is the regularization parameter.

To provide readers with a deeper understanding, we introduce two *explanations* of FCBC's functionality from other perspectives:

- FCBC can be further interpreted as an ensemble of classifiers operating on individual features. In other words, it functions as a feature-level ensemble of linear models, represented as $\mathcal{F}_{ij} = \{\mathcal{F}_{ij1}, \mathcal{F}_{ij2}, \ldots, \mathcal{F}_{iju_t}\}$, where $\mathcal{F}_{ijk}(\boldsymbol{x}_t) = w_{ijk,t} x_{t,k} + b_{ijk,t}$ and k denotes the k-th feature f_k. The prediction of $\mathcal{F}_{ij}$ is the average of the outputs from the feature-level base linear models corresponding to the features available in $\boldsymbol{x}_t$.
- FCBC can also be interpreted as an ensemble of two models, 1) a model that considers only the occurrence of features, and 2) a linear model with zero bias. The prediction of $\mathcal{F}_{ij}$ considers both the significance of a feature's occurrence and the contribution of its value to the classification process.

B. Model Adaptation

In EPAM, FCBC models are structured within an ensemble of ensemble framework. The inherent challenges include the dynamic imbalance between the positive and negative classes, as well as the uneven distribution within the negative class for each FCBC model.

For the j-th FCBC model $\mathcal{F}_{ij}$ for class c_i, the cumulative loss is:

$$Loss_{ij} = \sum_{t, y_t = c_i} r_{ij,t} l_{ij,t} + \sum_{t, y_t \neq c_i} r_{ij,t} l_{ij,t}, \tag{31}$$

where $l_{ij,t}$ is the loss value of $\mathcal{F}_{ij}$ on the instance $(\boldsymbol{x}_t, y_t)$ at time t, and $r_{ij,t}$ is the loss weight. The first component represents the loss on instances from the positive class, i.e. instances from the class c_i. The second component accounts for the loss on instances from the negative class, i.e., instances from the remaining classes $c_k (k \neq i)$. Previous work, such as EEOF [2], treats the negative class as a single entity, applying the same loss weight to all instances belonging to the negative class to address the imbalance between positive and negative classes. Specifically, in EEOF, (31) is transformed to:

$$Loss_{ij} = \sum_{t, y_t = c_i} r^+_t l_{ij,t} + \sum_{t, y_t \neq c_i} r^-_t l_{ij,t}, \tag{32}$$

where r^+_t and r^-_t are the loss weights for instances belonging to the positive and negative classes, respectively. To balance the loss feedback from the first and the second components in (32), the relationship between r^+_t and r^-_t is defined as $r^+_t \hat{p}^{(i)}_t = r^-_t (1 - \hat{p}^{(i)}_t)$, where $\hat{p}^{(i)}_t$ is the estimated prior probability of class c_i at time t. However, as analyzed earlier, the negative class is not a singular entity but comprises multiple distinct classes, i.e. $\{c_k | k \neq i\}$. Assigning a uniform weight r^-_t for different classes without accounting for their individual prior probabilities is insufficient, leading to inaccurate predictions in handling real-world data streams with class evolution.

In EPAM, the error feedback for each FCBC $\mathcal{F}_{ij}$ is initially balanced by treating the negative class as a set of distinct classes. The relationship between the loss weights for different classes

is defined as follows:

$$r_t^+ \hat{p}_t^{(i)} = \sum_{c_k, c_k \neq c_i} r_{k,t} \hat{p}_t^{(k)}, \tag{33}$$

where instances from the class $c_k (k \neq i)$ are assigned with a loss weight $r_{k,t}$. To balance the error feedback among classes that are designated as the negative class, the following condition must hold:

For any two classes $c_\alpha, c_\beta \in \{c_k | c_k \neq c_i\}$:

$$r_{\alpha,t} \hat{p}_t^{(\alpha)} = r_{\beta,t} \hat{p}_t^{(\beta)}. \tag{34}$$

Solving (33) and (34), we obtain:

$$r_{k,t} = \frac{r_t^+}{n_t - 1} \frac{\hat{p}_t^{(i)}}{\hat{p}_t^{(k)}}, \tag{35}$$

where n_t is the number of observed classes until time t. To introduce diversity within the group of FCBC models $\mathcal{F}_i = \{\mathcal{F}_{i1}, \mathcal{F}_{i2}, \ldots, \mathcal{F}_{im}\}$ for class c_i, the loss weight for $\mathcal{F}_{ij}$ is generated using the Poisson distribution, a common technique in online learning algorithms [41]. Setting $r_t^+ = 1$, the loss weight $r_{ij,t}$ for $\mathcal{F}_{ij}$ is generated by:

$$r_{ij,t} \sim \begin{cases} Poisson(1), & y_t = c_i \\ Poisson\left(\frac{\hat{p}_t^{(i)}}{(n_t - 1) * \hat{p}_t^{(k)}}\right), & y_t = c_k \neq c_i. \end{cases} \tag{36}$$

The model adaptation of EPAM is summarized in Algorithm 1. When an instance $(\boldsymbol{x}_t, y_t)$ arrives, the assigned binary label $y_{i,t}$ and the loss weight $r_{ij,t}$ are first computed for each FCBC $\mathcal{F}_{ij}$ (lines 3 to 10). Then, the weight and bias vectors in $\mathcal{F}_{ij}$ are updated (lines 11 to 13). If needed, an optional sparsity process is applied (lines 14 to 17).

C. Overview of EPAM in Action

An overview of EPAM in processing the data stream is displayed in Algorithm 2. In the data stream, EPAM first produces a prediction for each incoming instance, and then updates upon receiving the true label. The prediction process (line 8) employs the activation/deactivation strategy of OVA models in response to class reoccurrence and class disappearance, as well as the confidence-triggered fallback mode proposed in EEOF. The implementation details can be found in [2]. When the true label y_t arrives, EPAM updates the class status (line 12) following the strategy adopted in [1], [2]. Specifically, if the estimated prior probability of a class c_i is less than a predefined threshold, i.e. $\hat{p}_t^{(i)} < 10^{-5}$, class c_i is regarded as a disappeared class. If the true label y_t corresponds to a novel class, m FCBC models are initialized with weight and bias vectors filled with 0s in the feature space $\mathcal{X}_t$ of $\boldsymbol{x}_t$ (line 14). These models are subsequently incorporated into the EPAM model $\mathcal{F}$ (line 15). In addition, the prior probability for this novel class is initialized (lines 16 and 17). At the end of each update process, the estimated prior probability vector $\hat{\boldsymbol{p}}_t$ and the EPAM model $\mathcal{F}$ are updated (lines 19 and 20).

Algorithm 1: Adaptation of EPAM.

Input: Instance $(\boldsymbol{x}_t, y_t)$; EPAM model $\mathcal{F} = \{\mathcal{F}_{ij}\}$; Estimated prior probability vector $\hat{\boldsymbol{p}}_t = [\hat{p}_t^{(i)}]$; Number of seen classes n_t; Tradeoff parameters C and A; Whether to process the sparsity steps $sparsity$; Regularization parameter λ; Proportion of selected parameters B.
Output: Updated $\mathcal{F}$.

1: **for** $i \leftarrow 1 : n_t$ **do**
2: **for** $j \leftarrow 1 : m$ **do**
3: **if** $y_t == c_i$ **then**
4: $r_{ij,t} \sim Poisson(1)$
5: $y_{i,t} \leftarrow +1$
6: **else**
7: $k \leftarrow Find(k | c_k = y_t)$
8: $r_{ij,t} \sim Poisson(\hat{p}_t^{(i)} / ((n_t - 1) * \hat{p}_t^{(k)}))$
9: $y_{i,t} \leftarrow -1$
10: **end if**
11: Identify the common, disappeared, novel feature spaces
12: Calculate $\tau_{ij,t}$ using (26)
13: Update $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ in $\mathcal{F}_{ij}$ using (27), (28)
14: **if** $sparsity$ is True **then**
15: Project $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ to a L_1 ball using (29), (30)
16: Truncate $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$ by remaining a ratio of B elements with largest absolute values
17: **end if**
18: **end for**
19: **end for**

Algorithm 2: An Overview of EPAM in Learning Data Stream.

1: $\mathcal{F} \leftarrow \emptyset$, $\hat{\boldsymbol{p}}_t \leftarrow \emptyset$ // Initialize EPAM model and estimated prior probability vector
2: $t \leftarrow 1$
3: **while** Instance $\boldsymbol{x}_t$ arrives **do**
4: *// Predict*
5: **if** $t == 1$ **then**
6: $\hat{y}_t \leftarrow 0$ *// First instance in data stream*
7: **else**
8: Predict $\hat{y}_t$ using $\mathcal{F}$ *// Same method in EEOF*
9: **end if**
10: *// Update*
11: Receive true label y_t
12: Determine whether y_t is a novel, disappeared, or reoccurred class.
13: **if** y_t is a novel class **then**
14: Initialize m FCBC models $\mathcal{F}_{novel} = \{\mathcal{F}_{ij} | c_i = y_t, 1 \leq j \leq m\}$ for class y_t
15: $\mathcal{F} \leftarrow \mathcal{F} \cup \mathcal{F}_{novel}$
16: Initialize $\hat{p}_{novel}$ for class y_t
17: $\hat{\boldsymbol{p}}_t \leftarrow \hat{\boldsymbol{p}}_t \cup \{\hat{p}_{novel}\}$
18: **end if**
19: Update $\hat{\boldsymbol{p}}_t$ with the occurrence of y_t using (8)
20: Update $\mathcal{F}$ using Algorithm 1
21: **end while**

TABLE I
CHARACTERISTICS OF DATA STREAMS USED IN EXPERIMENTS

Data Stream	Instances	Features	Classes	Class Evolution Types	Feature Evolution Types
Letter(EC)	4500	16	3	Class Emergence	Capricious
Letter(DC)				Class Disappearance and Reoccurrence	Capricious
Letter(ET)				Class Emergence	Trapezoidal
Letter(DT)				Class Disappearance and Reoccurrence	Trapezoidal
Statlog(EC)		36		Class Emergence	Capricious
Statlog(DC)				Class Disappearance and Reoccurrence	Capricious
Statlog(ET)				Class Emergence	Trapezoidal
Statlog(DT)				Class Disappearance and Reoccurrence	Trapezoidal
Covertype(EC)		54		Class Emergence	Capricious
Covertype(DC)				Class Disappearance and Reoccurrence	Capricious
Covertype(ET)				Class Emergence	Trapezoidal
Covertype(DT)				Class Disappearance and Reoccurrence	Trapezoidal
KDDCUP99(C)	494021	117	23	Class Emergence, Disappearance and Reoccurrence	Capricious
KDDCUP99(T)					Trapezoidal
Poker-hand(C)	829201	25	10		Capricious
Poker-hand(T)					Trapezoidal
UNSW-NB15(C)	1087136	199	10		Capricious
UNSW-NB15(T)					Trapezoidal
Tweet Stream - A	36865	242	3	Class Emergence	Capricious
Tweet Stream - B	13926	244	3	Class Disappearance and Reoccurrence	Capricious
Tweet Stream - C	63285	242	4	Class Emergence	Capricious
Tweet Stream - 20 classes	135623	524	20	Class Emergence, Disappearance and Reoccurrence	Capricious

* The letters in $(\cdot)$ denote the synthetic properties. E: class emergence; D: class disappearance and reoccurrence; C: capricious; T: trapezoidal. For example, Letter(EC) is the data stream with synthetic class emergence for class evolution and synthetic capricious for feature evolution. KDDCUP99(C) is the data stream with synthetic capricious for feature evolution, while already exhibiting inherent class evolution. Tweet Stream - A/B/C/20 classes are real-world data streams with inherent class evolution and feature evolution.

VI. EXPERIMENTAL STUDY

In this section, we conduct a comprehensive experimental study to demonstrate the superiority of EPAM in addressing the problem of online learning in the open data space. These experiments are designed to answer the following research questions (RQs):

- RQ1. Why is it necessary to develop online learning methods for handling the open data space? Do the constructed online learning methods show better performance compared to the method that rely on extensive data storage for model adaptation?
- RQ2. Does EPAM outperform the baseline methods constructed from state-of-the-art online learning approaches based on either class evolution or feature evolution?
- RQ3. Do novel components in EPAM improve performance in learning real-world data streams from the open data space?

For RQ1 and RQ2, we evaluate models on a variety of synthetic and real-world data streams. For RQ3, we conduct an ablation study to assess the significance of different components in EPAM on handling real-world data streams. Furthermore, we evaluate the efficiency of EPAM on both synthetic and real-world data streams, and conduct a comprehensive sensitivity analysis of the hyperparameters.

A. Experimental Settings

1) Data Streams: Based on the synthetic properties, the data streams used in the following experiments are categorized into three types:

- Synthetic class evolution and synthetic feature evolution
- Real-world class evolution and synthetic feature evolution
- Real-world data streams in the open data space.

For the 1st type, we generate data streams from three tabular datasets: Letter Recognition [42], Statlog (Landsat Satellite), and Covertype [43]. For the 2nd type, we select three real-world data streams that exhibit class evolution: KDDCUP99 [44], Poker-hand [45], and UNSW-NB15 [46]. We then simulate feature evolution within these streams. For the 3rd type, we select four preprocessed data streams of TwitterCrawl data stream [47]: Tweet Stream - A/B/C/20 classes, as outlined in [1]. In these data streams, each instance represents a tweet, with the label assigned based on the hashtag describing its topic. Each feature corresponds to an individual word. In [3], [9], Tweet data is also utilized to investigate the coexistence of class evolution and feature evolution. While studies on class evolution [1], [2] also use Tweet data, they assume that all dimensions are available from the outset of the data stream. However, it is impossible to ascertain which words will emerge in the subsequent continuously generated tweets. For generating class evolution, we adopt the same approach as in the former work [1]. For generating feature evolution, we simulate the capricious scenario as in the former work [11], in which feature spaces change randomly, and the trapezoidal scenario as in the former work [17], in which the number of features monotonously increases. The key characteristics of all data streams are displayed in Table I.

2) Evaluation Metrics: The evaluation of models needs to account for class evolution. Same as the previous work [2], we apply the G-mean using sliding windows to monitor the performance of multi-class classification, and average these results to assess the overall performance on the data stream. The average of G-mean using sliding windows on the data stream with a total length of N is defined as $\frac{1}{N-s+1}\sum_{i=s}^{N} G_s(i)$, where $G_s(t) = G - mean(\{(y_i, \hat{y}_i)\}_{t-s+1}^{t})$, and s is the window size set to 200. Ten independent runs are conducted for each model

TABLE II
THE AVERAGE OF G-MEAN (MEAN/STD) OF MODELS OVER 10 RUNS ON DATA STREAMS WITH OPEN DATA SPACE

Data Stream	EPAM	EEOF-OLI^{2}DS$_1$	EEOF-OLI^{2}DS$_2$	EEOF-OL$_{SF}$	MCM
Letter(EC)	**0.7015/0.0105**	0.4398/0.0174	0.4964/0.0127	0.5174/0.0226	0.2047/0.0206
Letter(DC)	**0.7412/0.0143**	0.4618/0.0198	0.5345/0.0222	0.5469/0.0249	0.3990/0.0156
Letter(ET)	**0.8326/0.0471**	0.8122/0.0508	0.8141/0.0432	0.8146/0.0435	0.3744/0.0283
Letter(DT)	**0.8619/0.0302**	0.8106/0.0480	0.8367/0.0431	0.8372/0.0426	0.6309/0.0900
Statlog(EC)	**0.8731/0.0140**	0.5128/0.0150	0.5525/0.0166	0.5633/0.0160	0.2905/0.0153
Statlog(DC)	**0.9118/0.0174**	0.5343/0.0217	0.5942/0.0250	0.5992/0.0244	0.5694/0.0306
Statlog(ET)	**0.9013/0.0153**	0.7746/0.0234	0.7649/0.0187	0.7650/0.0187	0.4262/0.0040
Statlog(DT)	**0.9447/0.0114**	0.7919/0.0240	0.7973/0.0323	0.7974/0.0324	0.8414/0.0444
Covertype(EC)	**0.5636/0.0110**	0.5323/0.0159	0.5502/0.0120	0.5626/0.0103	0.2155/0.0101
Covertype(DC)	0.5636/0.0160	0.5068/0.0118	0.5587/0.0073	**0.5684/0.0142**	0.4401/0.0397
Covertype(ET)	**0.6476/0.0448**	0.6227/0.0410	0.6455/0.0351	0.6304/0.0415	0.2729/0.0183
Covertype(DT)	**0.6328/0.0478**	0.5952/0.0407	0.6286/0.0252	0.6261/0.0273	0.4968/0.0833
KDDCUP99(C)	**0.1747/0.0038**	0.0837/0.0027	0.0801/0.0023	0.0683/0.0027	0.0186/0.0006
KDDCUP99(T)	**0.2951/0.0490**	0.2324/0.0351	0.0968/0.0158	0.0892/0.0130	0.1676/0.0572
Poker-hand(C)	**0.2866/0.0025**	0.2194/0.0030	0.2551/0.0035	0.2253/0.0043	0.0350/0.0010
Poker-hand(T)	**0.3678/0.0289**	0.2625/0.0168	0.1531/0.0138	0.2648/0.0265	0.0556/0.0133
UNSW-NB15(C)	**0.8443/0.0004**	0.8405/0.0007	0.8397/0.0005	0.8405/0.0007	0.8297/0.0000
UNSW-NB15(T)	**0.8380/0.0058**	0.8325/0.0082	0.8255/0.0080	0.8260/0.0110	0.8345/0.0044
Tweet Stream - A	**0.6967/0.0058**	0.6809/0.0000	0.6674/0.0070	0.6804/0.0080	0.2014/0.0043
Tweet Stream - B	**0.7920/0.0021**	0.7792/0.0000	0.7916/0.0028	0.7846/0.0015	0.2472/0.0246
Tweet Stream - C	**0.6593/0.0053**	0.6019/0.0000	0.6356/0.0054	0.6542/0.0066	0.1852/0.0067
Tweet Stream - 20 classes	**0.0723/0.0014**	0.0708/0.0000	0.0451/0.0018	0.0686/0.0016	0.0000/0.0000
Win/Tie/Loss	-	0/4/18	0/6/16	0/7/15	0/1/21

* The highest mean results are highlighted in bold. The Win/Tie/Loss counts represent the number of cases in which the selected model significantly outperforms, shows no significantly difference, or performs significantly worse compared with EPAM, respectively (Wilcoxon rank sum test at 95% confidence level).

TABLE III
TUNED HYPERPARAMETERS VIA GRID SEARCH OF EPAM

Dataset	A	B	C	sparsity
Letter(EC)	10	-	1	False
Letter(DC)	5	-	1	False
Letter(ET)	10	-	1	False
Letter(DT)	10	-	1	False
Statlog(EC)	5	-	1	False
Statlog(DC)	5	-	1	False
Statlog(ET)	50	-	1	False
Statlog(DT)	10	-	1	False
Covertype(EC)	1000	-	0.1	False
Covertype(DC)	100	-	1	False
Covertype(ET)	50	-	10	False
Covertype(DT)	50	-	1	False
KDDCUP99(C)	500	-	10	False
KDDCUP99(T)	50	-	1	False
Poker-hand(C)	5	0.6	0.1	True
Poker-hand(T)	1	-	0.1	False
UNSW-NB15(C)	500	-	1	False
UNSW-NB15(T)	1000	-	1000	False
Tweet Stream - A	1	-	1	False
Tweet Stream - B	5	-	1	False
Tweet Stream - C	1	-	1	False
Tweet Stream - 20 classes	100	0.6	10	True

on each data stream, and the significance of differences between algorithms is evaluated using the Wilcoxon rank-sum test [48].

3) Compared Approaches: To the best of our knowledge, only two models, DXMiner [9] and MCM [3], considered the open data space in a chunk-by-chunk manner, with none of them in an online learning manner. The latter work MCM is selected for comparison. We also compare EPAM with aforementioned baseline models: EEOF-OLI^{2}DS$_1$ and EEOF-OLI^{2}DS$_2$, which integrate state-of-the-art techniques for either class evolution or feature evolution, along with the baseline model EEOF-OL$_{SF}$. To ensure a fair comparison, the hyperparameters in MCM, EEOF-OLI^{2}DS$_1$, EEOF-OLI^{2}DS$_2$, and EEOF-OL$_{SF}$ are either set to the recommended values or determined by grid search within the suggested value ranges. EPAM adopts the same hyperparameter settings as EEOF-OL$_{SF}$, where the $A \in [1, 5, 10, 50, 100, 500, 1000]$, $B \in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]$, and $C \in [10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10^{1}, 10^{2}, 10^{3}, 10^{4}]$. The hyperparameter λ is set to 30, following prior work [16], [17]. For EPAM and baseline methods, whether to enable the sparsity strategy is determined via grid search rather than manual tuning. The tuned hyperparameters via grid search of EPAM are listed in Table III. The size of initial dataset for grid search and the chunk size for MCM are set to 1000. The instances in the initial dataset are used solely for model warm-up and hyperparameter selection, and are excluded from the evaluation process. Our source code is available at https://github.com/lflfdxfn/EPAM-Implementation.

B. Comparisons on Synthetic and Real-World Data Streams (RQ1/RQ2)

1) To Answer RQ1: The performance of EPAM and the compared models is presented in Table II. As observed in the columns for MCM, EEOF-OLI^{2}DS$_1$, EEOF-OLI^{2}DS$_2$ and EEOF-OL$_{SF}$, MCM generally underperforms in most cases, with better performance only on Statlog(DC), Statlog(DT), KDDCUP99(T), and UNSW-NB15(T). In addition, as indicated by the Win/Tie/Loss metric, MCM performs significantly worse than EPAM in 21 cases and performs comparably to EPAM on UNSW-NB15(T). In MCM, instances are gathered in chunks. In each chunk, a unified feature space is created for all instances within. Then a base classifier is trained with all observed features and classes in this chunk. However, this approach is unsuitable for managing the dynamic data stream with continuously evolving sets of features and classes. MCM requires substantial data collection before it can adapt to novel classes and novel features, leading to delays in tracking the latest distribution

in the data stream and resulting in suboptimal predictions. In particular, on the Tweet Stream - 20 classes, classes and features emerge abruptly and disappear rapidly, meaning those classes and features encountered in one chunk may disappear and never reoccur in subsequent chunks. MCM fails to achieve multi-class classification within any individual sliding window under the G-mean metric. Online learning models closely track the dynamics of class evolution and feature evolution, demonstrating their powerful performance on the dynamic data stream in the open data space.

2) To Answer RQ2: As observed in the columns for EPAM and other online learning models in Table II, the Win/Tie/Loss results indicate that EPAM demonstrates significantly superior performance in most cases and comparable performance in a few. Specifically, compared to the highest results among the baseline online learning models, EPAM exhibits substantial improvements, achieving gains of 35.6% in Letter(EC), 35.5% in Letter(DC), 55.0% in Statlog(EC), 52.2% in Statlog(DC), 12.3% in Poker-hand(C), and 109% in KDDCUP99(C). All aforementioned data streams have the capricious type of feature evolution. We attribute these significant improvements to the employment of FCBC in EPAM. In capricious data streams, features emerge and disappear randomly. The integration of the proposed adaptive bias vector in FCBC offers a more comprehensive solution by catering a multitude of potential feature combinations and identifying the optimal decision boundary, thereby enhancing the multi-class classification performance of EPAM. The Covertype data streams, namely Covertype(EC/DC/ET/DT), are special cases where linear models with zero biases effectively separate classes. For these data streams, the parameter A with a higher value that emphasizes the update of the weight vector in the objective function (15), is selected via grid search on the initial offline dataset. This helps prevent EPAM from overfitting, resulting in comparable or slightly improved performance. In trapezoidal data streams such as Letter(ET/DT) and Statlog(ET/DT), EPAM still demonstrates notable improvements, albeit to a lesser degree.

The primary distinction between EEOF-OLI^{2}DS$_1$ and EEOF-OLI^{2}DS$_2$ lies in the generation of the loss weight. A comparison between these models reveals that the time-decayed dynamic class ratio in (8) is more effective than the cumulative class ratio in (10) for tracking dynamic class imbalance. However, models employing the time-decayed dynamic class ratio, specifically EEOF-OLI^{2}DS$_2$ and EEOF-OL$_{SF}$, both fail on KDDCUP99(T). In KDDCUP99, there are many tiny classes with the tiniest class containing only two instances across the whole data stream. For a small class c_k, it is treated as the negative class for OVA classifiers in $\mathcal{F}_i(i \neq k)$. In models with the time-decayed dynamic class ratio, instances of class c_k tend to provide lower loss feedback for $\mathcal{F}_i(i \neq k)$, as their loss weights decrease in conjunction with the other classes designated as the negative class. With inadequate loss feedback from the small class c_k designated as the negative class, instances are more prone to being misclassified as class c_k, resulting in suboptimal performance in multi-class classification. The model EEOF-OLI^{2}DS$_1$ achieves better performance, however, it compromises the recalls of small classes. The cumulative class ratio struggles to adapt to dynamic class ratios for small classes, and the OVA model $\mathcal{F}_k$ for the small class c_k will receive excessive loss feedback from the negative class. Consequently, fewer or even no instances, are predicted as class c_k, leading to a significant decrease in the recall of class c_k. The critical factor here is the appropriate assignment of loss weights for different classes designated as the negative class. EPAM addresses this problem through its innovative model adaptation strategy in (35) on balancing among classes designated as the negative class, achieving better performance in multi-class classification.

To visually demonstrate the effectiveness of EPAM, Fig. 4 illustrates the performance trends of these algorithms on real-world data streams with open data space, namely Tweet Stream - A/B/C/20 classes. In these data streams, different topics are selected as the classes of interest, while textual words are treated as features [1]. Novel topics and words may emerge, existing ones may gradually fall out of use, and disappeared ones may regain popularity. The goal is to categorize each tweet into the correct topic. Notably, the initial dataset used for grid search in each Tweet Stream contains 1000 samples. In each successive data stream, besides features and classes in the initial dataset, there are 1 novel class and 12 novel features for Tweet Stream - A, 0 novel classes and 12 novel features for Tweet Stream - B, 2 novel classes and 15 novel features for Tweet Stream - C, and 6 novel classes and 89 novel features for Tweet Stream - 20 classes. EPAM achieves better performance as the amount of instances increases, whereas the other four algorithms converge to lower values. We attribute this improvement to the employment of FCBC and the innovative model adaptation strategy in EPAM.

C. Ablation Study (RQ3)

To assess the importance of each novel component proposed in EPAM for handling real-world data streams, we conduct an ablation study from two perspectives: 1) the model adaptation approach and 2) the integrated OVA classifier. Specifically, we investigate four variants of EPAM, as displayed in Table V.

For clarity and conciseness, the following abbreviations are adopted: *Ori.* (Original) denotes the original version of either the novel model adaptation method or the novel OVA classifier (FCBC) in EPAM. *Singular* refers to the model adaptation method in the state-of-the-art method in online learning with class evolution, EEOF [2], as defined in (9), in which classes designated as the negative class are treated as a singular entity. *Static* represents the model adaptation method in the state-of-the-art method in online learning with feature evolution, OLI^{2}DS [16], which employs the cumulative class ratio, as defined in (10). *Zero-bias* and *Scalar-bias* correspond to the variants of FCBC in which the feature contributed bias vector is eliminated, and the bias term is assigned with a value of 0 or a scalar value for diverse feature spaces, respectively.

As shown in the columns for EPAM and EPAM-A1 in Table IV, EPAM achieves significantly better performance across all data streams. Both methods monitor the dynamic class imbalance using the time-decayed approach. However, they differ in whether they address the imbalance issue among classes

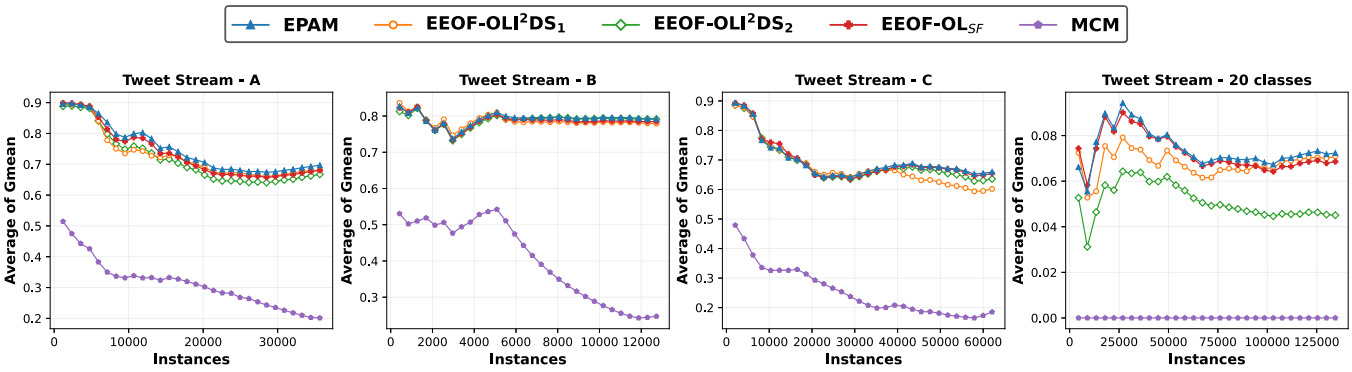


Fig. 4. Trends of the average of G-mean of EPAM and competing methods over 10 runs on Tweet Stream - A/B/C/20 classes.

TABLE IV
THE AVERAGE OF G-MEAN (MEAN/STD) OF EPAM AND ITS VARIANTS OVER 10 RUNS ON REAL-WORLD DATA STREAMS

Data Stream	EPAM	EPAM-A1	EPAM-A2	EPAM-C1	EPAM-C2
KDDCUP99(C)	**0.1747/0.0038**	0.0976/0.0030	0.1145/0.0024	0.1228/0.0029	0.1650/0.0036
KDDCUP99(T)	0.2951/0.0490	0.1141/0.0124	0.1878/0.0270	0.2807/0.0525	**0.2964/0.0480**
Poker-hand(C)	**0.2866/0.0025**	0.2071/0.0088	0.1792/0.0015	0.1979/0.0015	0.2304/0.0011
Poker-hand(T)	**0.3678/0.0289**	0.2618/0.0187	0.1774/0.0152	0.3245/0.0324	0.3601/0.0269
UNSW-NB15(C)	**0.8443/0.0004**	0.8412/0.0005	0.8406/0.0006	0.8423/0.0004	0.8425/0.0004
UNSW-NB15(T)	**0.8380/0.0058**	0.8319/0.0072	0.8349/0.0049	0.8344/0.0048	0.8379/0.0058
Tweet Stream - A	**0.6967/0.0058**	0.6788/0.0059	0.6848/0.0000	0.6961/0.0068	0.6763/0.0088
Tweet Stream - B	**0.7920/0.0021**	0.7851/0.0011	0.7753/0.0000	0.7890/0.0021	0.7906/0.0033
Tweet Stream - C	**0.6593/0.0053**	0.6542/0.0045	0.6055/0.0000	0.6574/0.0019	0.6568/0.0072
Tweet Stream - 20 classes	0.0723/0.0014	0.0672/0.0013	0.0228/0.0000	**0.0728/0.0016**	0.0203/0.0009
Win/Tie/Loss	-	0/0/10	0/1/9	0/5/5	0/5/5

* The highest mean results are highlighted in bold. The Win/Tie/Loss counts represent the number of cases in which the selected model significantly outperforms, shows no significantly difference, or performs significantly worse compared with EPAM, respectively (Wilcoxon rank sum test at 95% confidence level).

TABLE V
COMPONENTS IN EPAM AND ITS VARIANTS FOR ABLATION STUDY

Algorithm	Model Adaptation			OVA Classifier		
	Ori.	*Singular*	*Static*	*Ori.*	*Zero-bias*	*Scalar-bias*
EPAM	✓	-	-	✓	-	-
EPAM-A1	-	✓	-	✓	-	-
EPAM-A2	-	-	✓	✓	-	-
EPAM-C1	✓	-	-	-	✓	-
EPAM-C2	✓	-	-	-	-	✓

designated as the negative class, which we regard as the main factor contributing to the observed improvements. Compared with the variant EPAM-A2, EPAM also achieves significantly better or comparable performance. Although the cumulative class ratio method adopted in EPAM-A2 significantly outperforms the time-decayed dynamic class ratio employed in EPAM-A1 on KDDCUP99 data streams, this method compromises the recalls of minority classes, which is consistent with our previous findings. In contrast, the novel model adaptation method in EPAM takes all classes into account, thereby demonstrating its superiority in the task of multi-class classification. The comparison between EPAM and its two variants, EPAM-C1 and EPAM-C2, further demonstrates that the feature contributed bias enhances performance, yielding results that are comparable or superior to those of the variants. Specifically, EPAM achieves remarkable improvements of 42.3% on KDDCUP99(C), 44.8% on Poker-hand(C) compared with EPAM-C1, and improvements of 24.3% on Poker-hand(C), 3.02% on Tweet Stream - A, 256% on Tweet Stream - 20 classes, over EPAM-C2. In summary, novel components of EPAM contribute positively and significantly to the EPAM's superior performance.

D. Efficiency Analysis

As shown in Algorithm 2, the procedure of EPAM consists of a prediction process and an update process for each incoming data instance $\boldsymbol{x}_t \in \mathbb{R}^{d_t}$. At time t, define u_t and n_t as the number of observed features and classes up to time t, respectively. For the prediction process, according to (4), the time complexity of the calculation in line 8 is $O(mn_td_t)$, where m denotes the ensemble size. For the update process, the time complexity of updating the estimated prior probability vector $\hat{\boldsymbol{p}}_t$ (line 19) is $O(n_t)$. As illustrated in Algorithm 1, for each base OVA classifier $\mathcal{F}_{ij}$, the time complexity of identifying feature spaces (line 11) is $O(u_t)$. Additionally, the update of w_{ij} and b_{ij} (lines 12 and 13) and their sparsity processing (lines 14 to 17) result in a time complexity of $O(u_t + d_t)$. For the EPAM model $\mathcal{F} = \{\mathcal{F}_{ij}\}$, the time complexity of updating $\mathcal{F}$ is $O(mn_t(u_t + d_t))$. Combining the time complexities of the prediction and update processes, the total time complexity of EPAM is $O(mn_t(u_t + d_t))$ per instance.

The average runtime per instance of EPAM and competing methods over 10 runs on both synthetic and real-world data streams is illustrated in Fig. 5 . All experiments are performed on

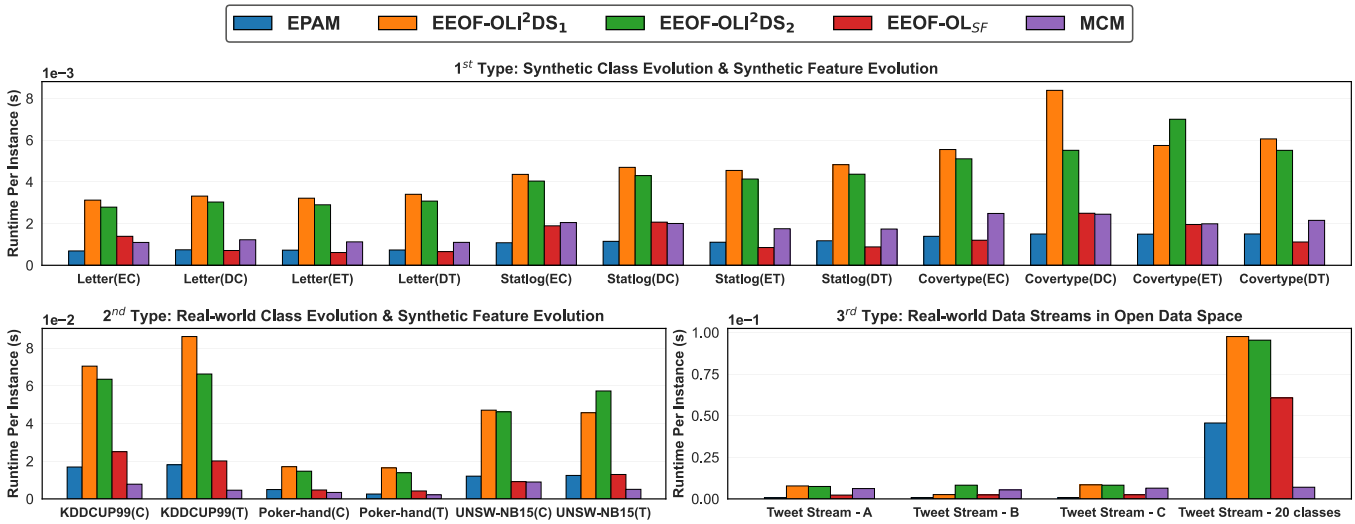


Fig. 5. The average of runtime per instance (in seconds) of EPAM and competing methods over 10 runs on data streams with open data space.

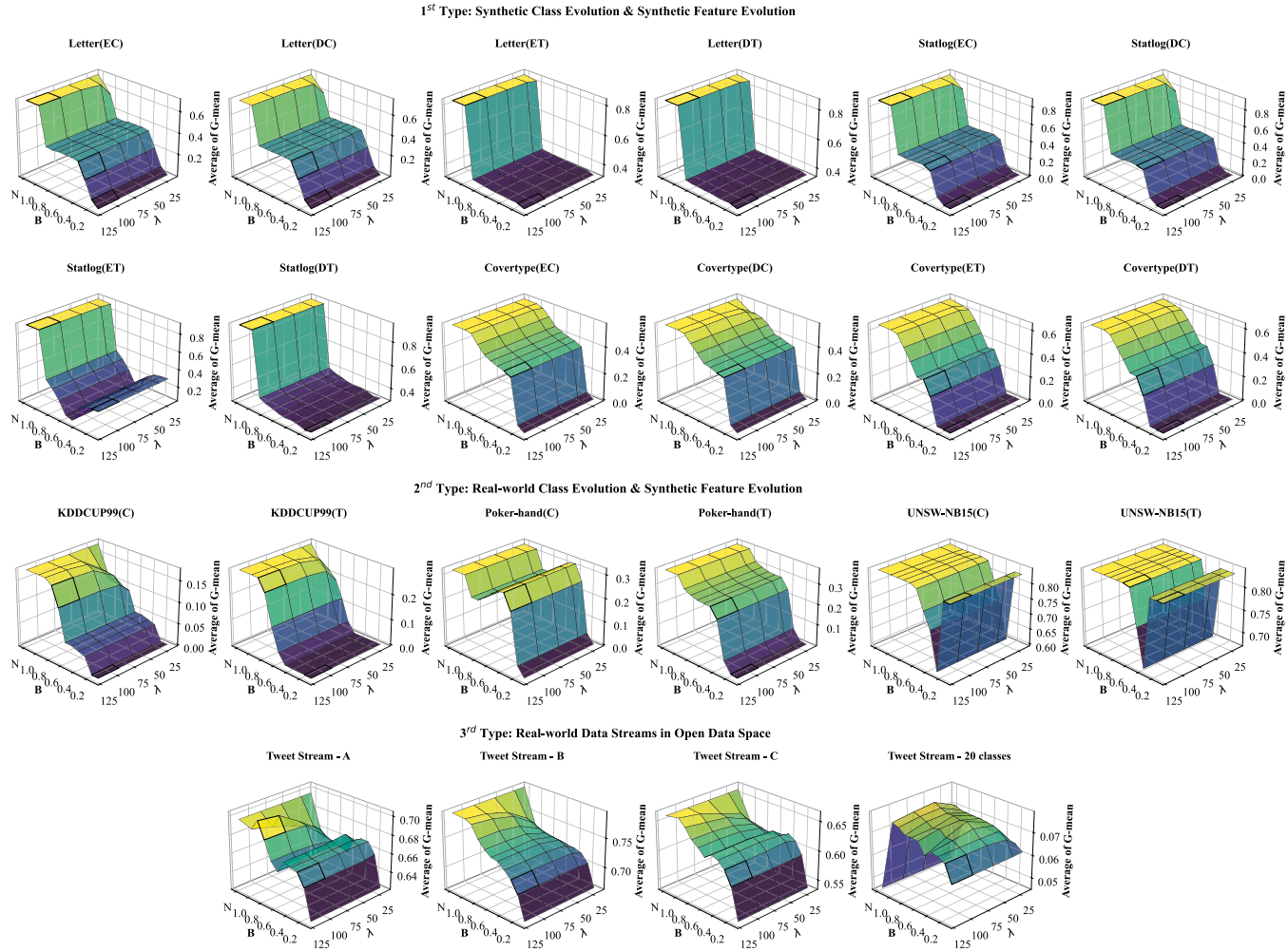


Fig. 6. The average of G-mean of EPAM with varying λ and B over 10 runs on data streams with open data space. The letter "N" indicates that the sparsity process is not applied.

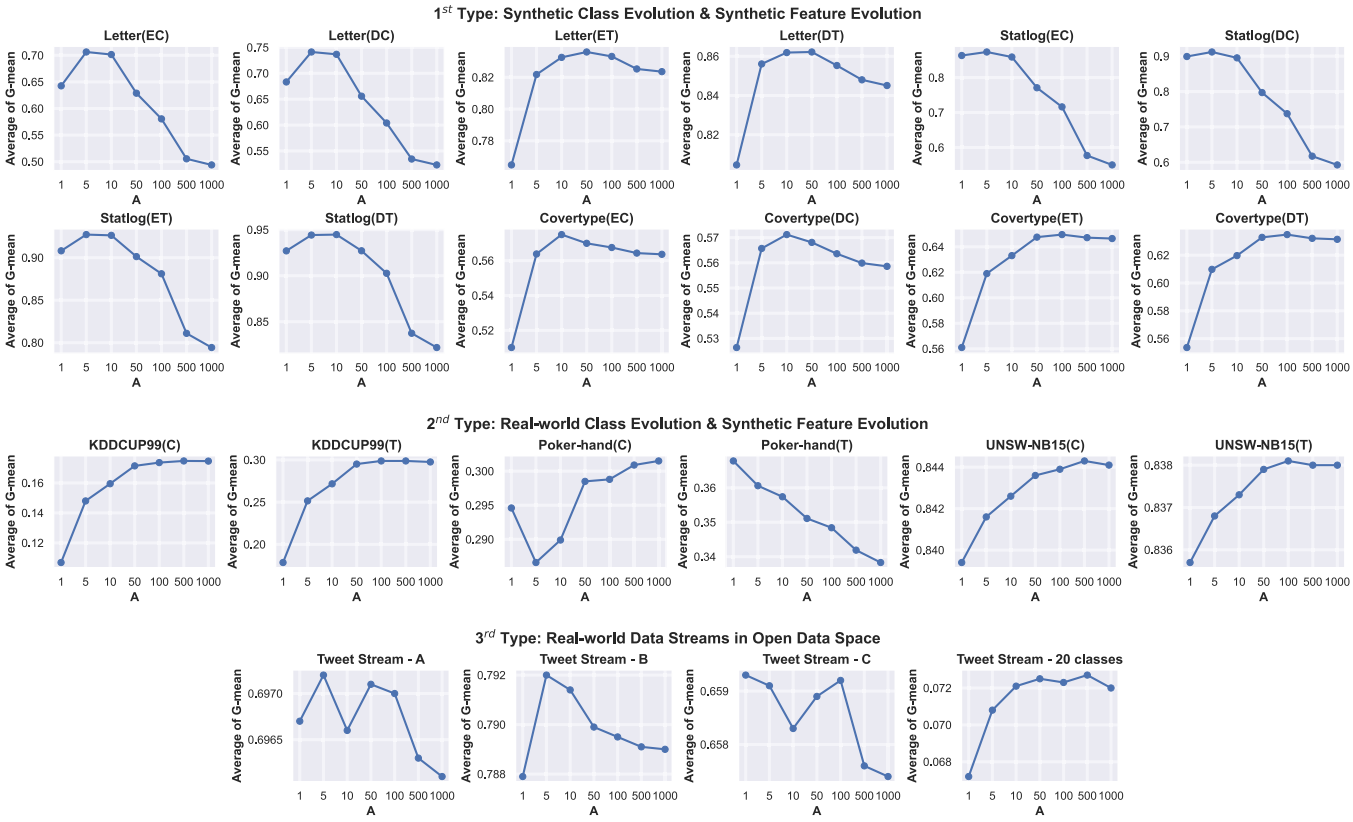


Fig. 7. The average of G-mean of EPAM with varying A over 10 runs on data streams with open data space.

an HPC cluster managed by SLURM, equipped with 100 Intel Xeon CPU cores running at 3.90 GHz and 1 TB of memory. Compared with the baseline methods, EPAM runs considerably faster than EEOF-OLI^{2}DS$_1$ and EEOF-OLI^{2}DS$_2$ in all data streams, while achieving comparable efficiency to EEOF-OL$_{SF}$. According to the complexity analysis in [16] and the algorithm described in [13], the time complexity of OLI^{2}DS and OL$_{SF}$ is $O(u_t + d_t)$. As a result, the baseline models, EEOF-OLI^{2}DS$_1$, EEOF-OLI^{2}DS$_2$, and EEOF-OL$_{SF}$, exhibit a time complexity of $O(mn_t(u_t + d_t))$, which is asymptotically equivalent to that of EPAM. MCM processes data streams in a chunk-by-chunk manner, yielding notably higher throughput, particularly in Tweet Stream - 20 classes. However, as shown in Table II, MCM fails to perform valid multi-class classification within any individual sliding window in this data stream. Accordingly, we conclude that EPAM is capable of efficiently handling both feature and class evolution simultaneously.

E. Sensitivity Analysis

A comprehensive sensitivity analysis is conducted on several key hyperparameters, namely the sparsity proportion B, the regularization parameter λ, and the trade-off parameter A, as illustrated in Figs. 6 and 7. Specifically, B denotes the proportion of retained parameters and determines the fraction of non-zero values in $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$, while λ constrains the maximum L_1-norm of $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$, as defined in (29), (30). The trade-off parameter A controls the update balance between the $\boldsymbol{w}_{ij,t}$ and $\boldsymbol{b}_{ij,t}$, as defined in objective function (15). In the experiments, B and A are varied over predefined grid search ranges, while λ is varied over $[15, 30, 60, 90, 120]$. The value "N" for B indicates that the sparsity process is not applied.

The impact of varying λ on EPAM is relatively limited compared with varying B. When $\lambda = 15$, performance degrades due to over-regularization induced by excessive sparsity, as observed on several data streams such as Letter(EC), Letter(DC), Statlog(EC), Statlog(DC), KDDCUP99(C), KDDCUP99(T), and Tweet Stream - A/B/C/20 classes. Particularly, in Tweet Stream - 20 classes, larger values of λ also lead to suboptimal performance, which we attribute to its high dimensionality. In this case, sparsity helps mitigate overfitting, and an appropriate choice of λ becomes crucial for achieving optimal performance. In contrast, B has a more significant impact on performance. When B is small and a large proportion of parameters are set to 0, excessive sparsity leads to unstable performance and limited model capacity of capturing informative patterns in the data. Although disabling sparsity generally improves performance, a notable exception is observed in the Tweet Stream (20 classes), where performance deteriorates without sparsity, again due to its high dimensionality.

The sensitivity of EPAM to A varies across different data streams. For Letter(EC/DC) and Statlog(EC/DC), as shown in Fig. 7, the performance of EPAM degrades to the level of baseline methods when A increases to 1000, where the performance of baseline methods is reported in Table II. At this point, the bias vector term is rarely updated and thus contributes

little to prediction. In their trapezoidal variants, namely Letter(ET/DT) and Statlog(ET/DT), the performance degradation is less pronounced but still noticeable. In capricious data streams, where features emerge and disappear randomly, the adaptive bias vector term plays a more critical role in handling diverse feature combinations and identifying appropriate decision boundaries. In contrast, trapezoidal data streams exhibit a monotonically increasing feature space, where the feature evolution is less drastic. In such cases, EPAM still benefits from the adaptive bias vector, although to a lesser extent. The Covertype data streams, namely Covertype(EC/DC/ET/DT), are special cases where linear models with zero biases can effectively separate classes. For these data streams, larger values of A, which emphasizes updates to the weight vector, are selected via grid search as listed in Table III. This helps reduce overfitting and leads to comparable or improved performance. Overall, the design of the trade-off parameter A enables EPAM to adapt to data streams with diverse characteristics, thereby improving its robustness.

In summary, it is advisable to employ grid search to determine appropriate values of B and A, thereby better adapting EPAM to specific data streams. The default value $\lambda = 30$ is recommended, consistent with prior work [16], [17]. In this study, we rely on grid search rather than manually tuning to determine whether to enable sparsity, ensuring a fair comparison across methods. For data streams in which the feature space evolves over time and is expected to expand to a high-dimensional regime, incorporating sparsity is generally recommended.

VII. Conclusion

In this work, we introduce EPAM, a novel model designed to address online learning in the open data space with both feature and class evolution. We approach this problem by constructing several baseline models grounded on state-of-the-art online learning techniques for either class evolution or feature evolution, analyzing their limitations, and highlighting the motivations for developing EPAM. In EPAM, a novel FCBC model is incorporated with an adaptive bias vector capable of adapting to diverse feature spaces. An innovative model adaptation strategy is designed to manage the imbalance among classes designated as the negative class for each FCBC model. Extensive experiments on synthetic and real-world data streams demonstrate the superiority of EPAM in handling both feature and class evolution effectively. However, as it is a linear model, the performance of FCBC may degrade in nonlinear separable cases. For instance, on the non-linearly separable real-world data stream Tweet Stream - 20 classes, although EPAM achieves significantly superior performance compared with other competing methods, its performance remains unsatisfactory. A future direction would be to design a nonlinear model for online learning in the open data space.

References

[1] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 6, pp. 1532–1545, Jun. 2016.

[2] Z. Cao, S. Zhang, and C.-T. Lin, "Online ensemble of ensemble OVA framework for class evolution with dominant emerging classes," in *Proc. IEEE Int. Conf. Data Mining*, 2023, pp. 968–973.

[3] M. M. Masud et al., "Classification and adaptive novel class detection of feature-evolving data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 7, pp. 1484–1497, Jul. 2013.

[4] S. Gu, Y. Qian, and C. Hou, "Incremental feature spaces learning with label scarcity," *ACM Trans. Knowl. Discov. Data*, vol. 16, no. 6, pp. 1–26, 2022.

[5] J. Dong, Y. Cong, G. Sun, T. Zhang, X. Tang, and X. Xu, "Evolving metric learning for incremental and decremental features," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 4, pp. 2290–2302, Apr. 2022.

[6] M. Mohammadi, A. Al-Fuqaha, S. Sorour, and M. Guizani, "Deep learning for IoT Big Data and streaming analytics: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2923–2960, Fourth quarter 2018.

[7] S. Wang, L. L. Minku, and X. Yao, "A systematic study of online class imbalance learning with concept drift," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 10, pp. 4802–4821, Oct. 2018.

[8] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neurocomputing*, vol. 459, pp. 249–289, 2021.

[9] M. M. Masud, Q. Chen, J. Gao, L. Khan, J. Han, and B. Thuraisingham, "Classification and novel class detection of data streams in a dynamic feature space," in *Proc. Mach. Learn. Knowl. Discov. Databases*, 2010, pp. 337–352.

[10] Y. He, C. Schreckenberger, H. Stuckenschmidt, and X. Wu, "Towards utilitarian online learning: A review of online algorithms in open feature space," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, 2023, pp. 6647–6655.

[11] Y. He, B. Wu, D. Wu, E. Beyazit, S. Chen, and X. Wu, "Online learning from capricious data streams: A generative approach," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2491–2497.

[12] Y. He, X. Yuan, S. Chen, and X. Wu, "Online learning in variable feature spaces under incomplete supervision," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 5, pp. 4106–4114.

[13] Q. Zhang, P. Zhang, G. Long, W. Ding, C. Zhang, and X. Wu, "Online learning from trapezoidal data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 10, pp. 2709–2723, Oct. 2016.

[14] E. Beyazit, J. Alagurajah, and X. Wu, "Online learning from data streams with varying feature spaces," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 3232–3239.

[15] L. Pan, C. Gao, J. Zhou, and J. Wang, "Learning with open-world noisy data via class-independent margin in dual representation space," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 6, pp. 6290–6298.

[16] D. You et al., "Online learning from incomplete and imbalanced data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10650–10665, Oct. 2023.

[17] Q. Zhang, P. Zhang, G. Long, W. Ding, C. Zhang, and X. Wu, "Towards mining trapezoidal data streams," in *Proc. IEEE Int. Conf. Data Mining*, 2015, pp. 1111–1116.

[18] K. Crammer, O. Dekel, J. Keshet, S. Shalev-Shwartz, and Y. Singer, "Online passive-aggressive algorithms," *J. Mach. Learn. Res.*, vol. 7, no. 19, pp. 551–585, 2006.

[19] E. Beyazit, M. Hosseini, A. Maida, and X. Wu, "Learning simplified decision boundaries from trapezoidal data streams," in *Proc. Artif. Neural Netw. Mach. Learn.*, 2018, pp. 508–517.

[20] Y. Liu, X. Fan, W. Li, and Y. Gao, "Online passive-aggressive active learning for trapezoidal data streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 6725–6739, Oct. 2023.

[21] S. Gu, T. Luo, M. He, and C. Hou, "Online learning with incremental feature space and bandit feedback," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12902–12916, Dec. 2023.

[22] H. Lian, Y. He, D. Wu, Z. Chen, X. Zhu, and X. Wu, "Online outlier detection in open feature spaces," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 10, pp. 6091–6106, Oct. 2025.

[23] B.-J. Hou, L. Zhang, and Z.-H. Zhou, "Learning with feature evolvable streams," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1–11.

[24] Z.-Y. Zhang, P. Zhao, Y. Jiang, and Z.-H. Zhou, "Learning with feature and distribution evolvable streams," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, vol. 119, pp. 11317–11327.

[25] J. Peng, J. Guo, Q. Yang, J. Lu, and J. Shao, "A general framework for mining concept-drifting data streams with evolvable features," in *Proc. IEEE Int. Conf. Data Mining*, 2021, pp. 1276–1281.

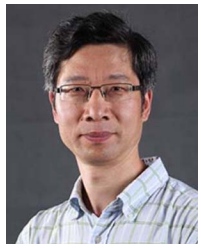
[26] L. L. Minku and X. Yao, "DDD: A new ensemble approach for dealing with concept drift," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 4, pp. 619–633, Apr. 2012.

[27] B.-J. Hou, L. Zhang, and Z.-H. Zhou, "Prediction with unpredictable feature evolution," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5706–5715, Oct. 2022.
[28] H. Lian, D. Wu, B.-J. Hou, J. Wu, and Y. He, "Online learning from evolving feature spaces with deep variational models," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 8, pp. 4144–4162, Aug. 2024.
[29] Y. He, B. Wu, D. Wu, E. Beyazit, S. Chen, and X. Wu, "Toward mining capricious data streams: A generative approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 3, pp. 1228–1240, Mar. 2021.
[30] S. Zhuo, D. Wu, Y. He, S. Huang, and X. Wu, "Online learning from mix-typed, drifted, and incomplete streaming features," *ACM Trans. Knowl. Discov. Data*, vol. 19, no. 8, pp. 1–28, 2025.
[31] D. You et al., "Online learning for data streams with incomplete features and labels," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4820–4834, Sep. 2024.
[32] S. Hashemi, Y. Yang, Z. Mirzamomen, and M. Kangavari, "Adapted one-versus-all decision trees for data stream classification," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 5, pp. 624–637, May 2009.
[33] S. U. Din et al., "Data stream classification with novel class detection: A review, comparison and challenges," *Knowl. Inf. Syst.*, vol. 63, no. 9, pp. 2231–2276, 2021.
[34] E. L. Allwein, R. E. Schapire, and Y. Singer, "Reducing multiclass to binary: A unifying approach for margin classifiers," *J. Mach. Learn. Res.*, vol. 1, pp. 113–141, 2001.
[35] R. Rifkin and A. Klautau, "In defense of one-vs-all classification," *J. Mach. Learn. Res.*, vol. 5, pp. 101–141, 2004.
[36] X. Xu, I. W. Tsang, and D. Xu, "Soft margin multiple kernel learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 24, no. 5, pp. 749–761, May 2013.
[37] S. Wang, L. L. Minku, and X. Yao, "A learning framework for online class imbalance learning," in *Proc. IEEE Symp. Comput. Intell. Ensemble Learn.*, 2013, pp. 36–45.
[38] S. Wang, L. L. Minku, and X. Yao, "Online class imbalance learning and its applications in fault detection," *Int. J. Comput. Intell. Appl.*, vol. 12, no. 04, 2013, Art. no. 1340001.
[39] S. Boyd and L. Vandenberghe, *Convex Optimization*. New York, NY, USA: Cambridge Univ. Press, 2004.
[40] J. Wang, P. Zhao, S. C. Hoi, and R. Jin, "Online feature selection and its applications," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 3, pp. 698–710, Mar. 2014.
[41] N. C. Oza and S. J. Russell, "Online bagging and boosting," in *Proc. 8th Int. Workshop Artif. Intell. Statist.*, 2001, pp. 229–236.
[42] D. Slate, "Letter recognition," UCI Machine Learning Repository, Dec. 31, 1990, doi: 10.24432/C5ZP40.
[43] J. Blackard, "Covertype," UCI Machine Learning Repository, Jul. 31, 1998, doi: 10.24432/C50K5N.
[44] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.
[45] R. Cattral, F. Oppacher, and D. Deugo, "Evolutionary data mining with automatic rule generalization," *Recent Adv. Comput., Comput. Commun.*, vol. 1, no. 1, pp. 296–300, 2002.
[46] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Secur. J.: A Glob. Perspective*, vol. 25, no. 1–3, pp. 18–31, 2016.
[47] R. Li, S. Wang, H. Deng, R. Wang, and K. C.-C. Chang, "Towards social user profiling: Unified and discriminative influence model for inferring home locations," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2012, pp. 1023–1031.
[48] J. Gibbons and S. Chakraborti, *Nonparametric Statistical Inference: Revised and Expanded*. Boca Raton, FL, USA: CRC Press, 2014.

Zhi Cao received the BSc degree in physics from the Southern University of Science and Technology, China, in 2017, and the PhD degree in computer science from the University of Technology, Sydney, Australia, in 2025. He is currently an AI engineer with Anhui Transport Consulting & Design Institute Company, Ltd, and a joint postdoctoral researcher with ATCDI and the University of Science and Technology of China. His research interests include machine learning, online learning, class evolution, ensemble learning, and data stream mining.

Peijia Qin received the BE degree in computer science and technology from the Southern University of Science and Technology, Shenzhen, China, in 2025. He is currently working toward the PhD degree in electrical and computer engineering with the University of California, San Diego, USA. He has authored or coauthored papers in areas including meta-learning, large language models (LLMs), LLM agents, and image generation on venues including ICML, TMLR, L4DC. He serves as a reviewer for top-tier artificial intelligence conferences, including ICLR, AAAI, NeurIPS, and ICML.

Chin-Teng Lin (Fellow, IEEE) received the Bachelor's of Science degree from the National Chiao-Tung University (NCTU), Taiwan, in 1986, and the master's and PhD degrees in electrical engineering from Purdue University, USA, in 1989 and 1992, respectively. He is currently a distinguished professor with the School of Computer Science and co-director of the Australian Artificial Intelligence Institute (AAII) within the Faculty of Engineering and Information Technology, University of Technology Sydney, Australia. He is also an honorary chair professor of electrical and computer engineering with NCTU. He is the coauthor of *Neural Fuzzy Systems* (Prentice-Hall) and the author of *Neural Fuzzy Control Systems with Structure and Parameter Learning* (World Scientific). His 953 publications include three books; 28 book chapters; 487 journal papers; and 435 refereed conference papers, including about 234 IEEE journal papers in the areas of neural networks, fuzzy systems, brain-computer interface, multimedia information processing, cognitive neuro-engineering, and human-machine teaming, that have been cited more than 45,500 times. His current h-index is 101, and his i10-index is 505. For his contributions to biologically inspired information systems, Prof Lin was awarded Fellowship with the IEEE, in 2005, and with the International Fuzzy Systems Association (IFSA), in 2012. He received IEEE Fuzzy Systems Pioneer Award, in 2017 and the IEEE Lotfi A. Zadeh Award for Emerging Technologies, in 2026. He has held notable positions as editor-in-chief of *IEEE Transactions on Fuzzy Systems* from 2011 to 2016; seats on Board of Governors for IEEE Circuits and Systems (CAS) Society (2005–2008), IEEE Systems, Man, Cybernetics (SMC) Society (2003–2005), IEEE Computational Intelligence Society (2008–2010); Chair of IEEE Taipei Section (2009–2010); Chair of IEEE CIS Awards Committee (2022, 2023); Distinguished Lecturer with IEEE CAS Society (2003–2005) and CIS Society (2015–2017); Chair of IEEE CIS Distinguished Lecturer Program Committee (2018–2019); Deputy editor-in-chief of *IEEE Transactions on Circuits and Systems-II* (2006–2008); program chair of IEEE International Conference on Systems, Man, and Cybernetics (2005); and general chair of the 2011 and 2027 IEEE International Conference on Fuzzy Systems.



Xin Yao (Fellow, IEEE) received the BSc degree from the University of Science and Technology of China (USTC) in 1982, the MSc degree from the North China Institute of Computing Technologies in 1985, and the PhD degree from USTC in 1990. He is currently the Tong Tin Sun chair professor of machine learning with Lingnan University, Hong Kong, China. His research interests include evolutionary computation, machine learning, and their real-world applications. He was the president (2014–2015) of IEEE CIS and editor-in-chief (2003–2008) of *IEEE Transactions on Evolutionary Computation*. His work won the 2001 IEEE Donald G. Fink Prize Paper Award; 2010, 2016 and 2017 IEEE Transactions on Evolutionary Computation Outstanding Paper Awards; 2011 IEEE Transactions on Neural Networks Outstanding Paper Award; 2010 BT Gordon Radley Award for Best Author of Innovation (Finalist); and other best paper awards at conferences. He received the 2012 Royal Society Wolfson Research Merit Award, 2013 IEEE CIS Evolutionary Computation Pioneer Award, and 2020 IEEE Frank Rosenblatt Award.